

Beijing University of Aeronautics and Astronautics



Regional Centre for Space Science and Technology Education in Asia and the Pacific

MATLAB Based Numerical Solution of Lambert Transfer Orbit

Final Report
Team Pilot Project
Micro-Satellite Technology

By

Neama MENSHAWI	**LS2525228**
Marwane OUFRID	**LS2525201**
Erick MAYTA	**LS2525215**
Fadlan ABU-SAFIAN	**LS2525230**
Carlos SAHUANAY	**LS2525221**

Venue: Hangzhou, China
Date: June 9th, 2026

Contents

Abbreviations v

Notation v

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 Historical Development . . . 3
- 1.3 Objectives . . . 6

2 Methodology 7
- 2.1 Solver 1: Solving a Typical Lambert Transfer Problem . . . 7
  - 2.1.1 Procedural Workflow . . . 7
  - 2.1.2 Solution for a Given Transfer Time . . . 8
- 2.2 Solver 2: Minimisation of Total Velocity Increment . . . 11
  - 2.2.1 Procedural Workflow . . . 12
  - 2.2.2 Transfer-Time Bounds . . . 12
  - 2.2.3 Cost Function . . . 13
  - 2.2.4 Method A: Grid Search with Local Refinement . . . 13
  - 2.2.5 Method B: Golden Section Search . . . 13
  - 2.2.6 Method C: Brent's Method (`fminbnd`) . . . 14
- 2.3 Solver 3: Lambert Transfer with J_2 Perturbation . . . 14
  - 2.3.1 Procedural Workflow . . . 15
  - 2.3.2 Two-Body Seed . . . 15
  - 2.3.3 Perturbed Equations of Motion . . . 16
  - 2.3.4 State Transition Matrix Propagation . . . 17
  - 2.3.5 RK4 Propagation of the Augmented State . . . 18
  - 2.3.6 Shooting Velocity Correction . . . 19
  - 2.3.7 Numerical Variants Used for Validation . . . 20

3 Results 23
- 3.1 Application Scenario . . . 23
- 3.2 Input Verification . . . 23
- 3.3 Solver 1 . . . 24
- 3.4 Solver 2 . . . 25
- 3.5 Solver 3 . . . 27
  - 3.5.1 Solver 3 at the Prescribed Transfer Time . . . 27
  - 3.5.2 Solver 3 at the Solver 2 Optimized Transfer Time . . . 27
  - 3.5.3 Solver 3 Numerical Method Comparison . . . 28
- 3.6 Overall Comparison of the Three Solvers . . . 29

4 Numerical Analysis and Validation 30
- 4.1 Tolerances implemented in the three Solvers . . . 30
- 4.2 Numerical Validation of lambert solver . . . 31
  - 4.2.1 Validation Test Cases of solver1 . . . 32
  - 4.2.2 Validation Test Cases of solver2 . . . 33
- 4.3 Numerical-Error Validation of Solver 3 . . . 34
- 4.4 Comparative analysis with established Lambert algorithms . . . 36

4.4.1 Reference Lambert Algorithms . . . 37
4.4.2 Time-of-Flight Sensitivity Analysis . . . 37
4.4.3 Transfer-Angle Sensitivity Analysis . . . 38
4.4.4 Near-180° Transfer . . . 39

5 Visualization **41**
5.1 Cross-Verification of Transfer Orbits . . . 41
5.2 Simulation via GMAT . . . 41
5.3 Simulation via STK . . . 42
5.4 Discussion of Cross-Verification . . . 43

6 Conclusion **44**

A Raw Numerical Validation Tables **47**

B Project Schedule **48**

C Work Distribution **48**

D Code Availability **49**

List of Figures

1 Geometric representation of Lambert's problem, Courtesy of Conte (2014). 1
2 Solver 1 Two-Body Lambert Problem Workflow. 7
3 Solver 2 Fuel Transfer Optimization Workflow. 12
4 Solver 3 workflow for the J_2-perturbed Lambert correction. 15
5 Geometric interpretation of one RK4 step applied to a component of Solver 3. 19
6 Orbit geometry of the application scenario defined by position vectors $\mathbf{r}_1$ and $\mathbf{r}_2$. 23
7 Plot of the three optimization methods for total ΔV as a function of TOF. 26
8 Solver 3 numerical variants 28
9 Total and component ΔV values for the two-body and J_2-corrected transfers. 29
10 Step-size convergence of the fixed-step propagation variants. 35
11 Solver 3 shooting-method convergence. 36
12 Combined velocity error versus time of flight. 38
13 Combined velocity error versus transfer angle. 39
14 Solver agreement for transfer angles approaching the critical 180° geometry. 40
15 GMAT visualisation of the transfer orbits: Solver 1 (top), Solver 2 (middle), Solver 3 (bottom). 41
16 STK visualisation of the transfer orbits: Solver1 (top), Solver2 (middle), Solver3 (bottom). 42

List of Tables

1 Comparison of Lambert's Problem Solvers. 3
2 Available MATLAB implementations of Lambert problem solvers. 6
3 Core parameters of the application scenario. 23
4 Numerical tolerances and limits used in the MATLAB code 30
5 Validation benchmarks with derivation and engineering effect 31
6 Solver1 validation results for five test cases. red=FAIL, orange=CHECK. . 32
7 Solver2 validation results for five test cases. red=FAIL, orange=CHECK. . 33
8 Solver3 STM validation tests. 35
9 Solver 3 comparison of augmented-state propagation methods for the optimized transfer case. 36
10 Comparison of Lambert algorithms 37
11 Orbital elements of the transfer orbits computed in GMAT 42
12 Orbital elements of the transfer orbits computed in STK 43
13 Raw numerical results of the TOF sensitivity analysis. 47
14 Raw numerical results of the transfer-angle sensitivity analysis. 47
15 Raw numerical results of the near-180° robustness analysis. 48
16 Team Pilot Project completion schedule. 48
17 Team Pilot Project work distribution. 48
18 Files included in the MATLAB code package for three lambert solvers. . . 49
19 Files included in the MATLAB code package for the comparative analysis. 49

Abbreviations

The following abbreviations are used throughout this report.

ABM4	Fourth-order Adams–Bashforth–Moulton method
GMAT	General Mission Analysis Tool
ODE	Ordinary differential equation
RK4	Classical fourth-order Runge–Kutta method
RK45	Adaptive fourth-/fifth-order Runge–Kutta method
STM	State transition matrix
STK	Systems Tool Kit
TOF	Time of flight

Notation

Vectors are written in bold type, while their corresponding magnitudes are written in italic type. Subscripts 1 and 2 refer to the departure and arrival points, respectively.

$\mathbf{r}_i$	Position vector at point i
$\mathbf{v}_i$	Velocity vector at point i
$\mathbf{v}_i^{(\mathrm{tr})}$	Transfer-orbit velocity at point i
$\mathbf{v}_{ci}$	Circular-orbit velocity at point i
$\Delta\mathbf{v}_i$	Velocity-increment vector at point i
$\Delta V_{\mathrm{total}}$	Total two-impulse velocity increment
Δt	Transfer time or time of flight
Δt_{opt}	Transfer time minimizing $\Delta V_{\mathrm{total}}$
$\Delta\theta$	Transfer angle
ΔE	Eccentric-anomaly difference
T_{tr}	Transfer-orbit period
μ	Gravitational parameter of the central body
R_E	Earth equatorial radius
J_2	Earth's second zonal harmonic coefficient
a	Semi-major axis
e	Orbital eccentricity
p	Semi-latus rectum
p_I, p_{II}	Limiting values of p
$\mathbf{h}$	Specific angular-momentum vector
$f, g, \dot{f}, \dot{g}$	Lagrange coefficients
$F(p)$	Lambert time-equation residual
$C(\Delta t)$	Transfer-time optimization cost function
$\mathbf{x}$	Cartesian state vector
$\mathbf{A}$	Dynamics Jacobian matrix
$\mathbf{\Phi}$	State transition matrix
$\mathbf{\Phi}_{rv}$	STM position–velocity sensitivity block
$\Delta\mathbf{r}^{(i)}$	Terminal-position residual at iteration i
$\Delta\mathbf{v}_1^{(i)}$	Initial-velocity correction at iteration i
S	Numerical integration step size
ε	Numerical tolerance

1 Introduction

Orbital mechanics provides the mathematical and physical rules that govern how spacecraft travel through space. Among the many problems in this field, the *Lambert transfer problem* has a special status (Hu, 2015): given two position vectors $\mathbf{r}_1$ and $\mathbf{r}_2$ relative to a central body, its gravitational parameter μ, and a prescribed transfer time Δt, find the two-body orbits - characterized by their initial velocity $\mathbf{v}_1$ and final velocity $\mathbf{v}_2$ — that connect the two points in exactly that time (Bate et al., 1971). This deceptively compact statement underlies launch vehicle design, rendezvous and docking of spacecraft, and in-orbit maneuver optimization (Curtis, 2019).

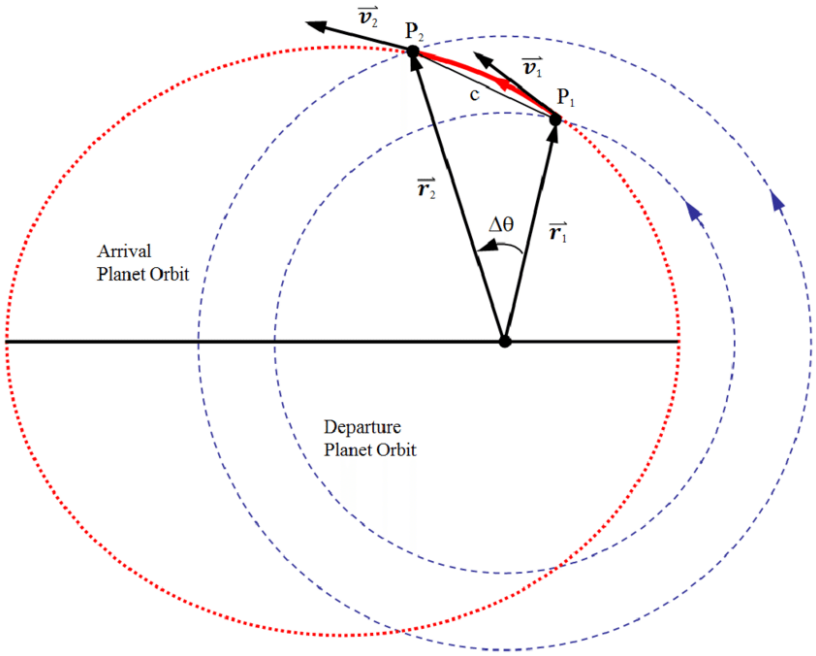


Figure 1: Geometric representation of Lambert's problem, Courtesy of Conte (2014).

The primary metric in any such transfer is the total velocity increment required at the two manoeuvre points, which is directly proportional to propellant consumption via the Tsiolkovsky rocket equation. Formally, the total ΔV for a two-impulse transfer departing from an initial orbit is (Kluever, 2018):

$$\Delta V_{\text{total}} = \left\| \mathbf{v}_1^{(\text{tr})} - \mathbf{v}_{1,\text{int}} \right\| + \left\| \mathbf{v}_{2,\text{arr}} - \mathbf{v}_2^{(\text{tr})} \right\|, \tag{1}$$

where $\mathbf{v}_{1,\text{int}}$ is the velocity on the initial orbit at departure and $\mathbf{v}_{2,\text{arr}}$ is the velocity on the arrival orbit. Minimizing ΔV_{total} over the transfer time Δt is therefore a fuel-optimization problem of direct engineering significance (Kluever, 2018).

1.1 Problem Statement

This project attempts to lower the difficulty of the problem by fixing a typical elliptical-transfer scenario in which two position vectors and the transfer time are given. A MATLAB program is required to obtain a numerical solution of the Lambert problem, including the calculation of geometric parameters, establishment of the time equation, determination

of the transfer orbit elements through numerical iteration, and calculation of the initial velocity using the Lagrange coefficients.

(i) Problem Conditions

Given the Earth's gravitational parameter

$$\mu = 3.986 \times 10^{14}\ \mathrm{m^3\,s^{-2}}, \tag{2}$$

the Earth's nonspherical perturbation coefficient

$$J_2 = 1.08263 \times 10^{-3}, \tag{3}$$

the initial position vector

$$\mathbf{r}_1 = \begin{bmatrix} 5000 & 10000 & 0 \end{bmatrix}^T \ \mathrm{km}, \tag{4}$$

the target position vector

$$\mathbf{r}_2 = \begin{bmatrix} 14000 & 5000 & 0 \end{bmatrix}^T \ \mathrm{km}, \tag{5}$$

and the transfer time

$$\Delta t = 3600\ \mathrm{s}. \tag{6}$$

(ii) Solving a Typical Lambert Transfer Problem

Assume that the initial and target orbits are both circular, the transfer orbit is elliptical, and the phase change along the transfer orbit is less than 180°. Consider the two-body model and design the corresponding transfer orbit for the following problems:

1. For the prescribed transfer time Δt, determine the semi-major axis a of the transfer orbit, the initial velocity increment $\Delta\mathbf{v}_1$, and the required velocity increment $\Delta\mathbf{v}_2$ upon arrival at the target position.
2. Determine the orbital transfer time Δt that minimizes the sum of the initial velocity increment Δv_1 and the required velocity increment Δv_2 at the target position under the given orbital-transfer conditions:

$$\min_{\Delta t} \left[\Delta v_1(\Delta t) + \Delta v_2(\Delta t)\right]. \tag{7}$$

(iii) Solving the Lambert Problem Considering J_2 Perturbation

The solution method for the Lambert problem considering perturbations can be divided into two steps:

1. **Solving for the initial values:** Based on the two-body model, construct the time-of-flight function $\Delta t(\Delta E)$ described by the eccentric-anomaly difference ΔE using the Keplerian Lagrange parameters. Obtain the solution within a single orbital period through numerical iteration. Finally, compute the initial and final velocity vectors using the state-transition relations.
2. **Solving for the accurate solution:** Incorporate the J_2 perturbation into the two-body model. Obtain the orbit and the state transition matrix through numerical integration. Finally, iteratively correct the initial velocity using a shooting method and obtain the accurate initial and final velocity vectors.

1.2 Historical Development

Johann Heinrich Lambert posed the problem in a 1761 letter to Euler (Lambert, 1761), and the first rigorous analytical proof was given by Lagrange (1780). Over the following two and a half centuries, the problem attracted contributions from Gauss (1809), Lancaster and Blanchard (1969), Avanzini (2008), and more recently Izzo (2015), among many others.

Across the literature, Lambert solvers can be interpreted through the same general sequence: the admissible Keplerian conics are represented by one scalar parameter, this parameter is inserted into a time-of-flight relation, and the resulting nonlinear equation is solved by iteration before the endpoint velocity vectors are recovered. What distinguishes one formulation from another is mainly the choice of four algorithmic elements (Izzo, 2015):

1. Iteration variable — the free parameter connected to the time-of-flight equation;
2. Iteration algorithm — the root-finding scheme;
3. Initial guess — the starter estimation procedure;
4. Terminal velocity reconstruction — how $\mathbf{v}_1$ and $\mathbf{v}_2$ are recovered.

The following table synthesizes the main characteristics of selected Lambert problem solvers, as reviewed and compared in the work of De la Torre Sangrà and Fantino (2015):

Table 1: Comparison of Lambert's Problem Solvers.

Reference	Iteration Variable	Iteration Algorithm	Initial Guess	Velocity Reconstruction
Lagrange (1780)	Semi-major axis a	Newton–Raphson	a_0 corresponding to minimum-energy orbit ($x = 0$)	Radial and transversal components
Gauss (1809)	Sector-to-triangle area ratio y	Gauss successive substitution (hypergeometric ${}_2F_1$)	$y_0 = 1$ (1:1 area ratio)	Gauss f and g functions; singular at $\theta = \pi$
Lancaster and Blanchard (1969)	Universal variable $x = \sqrt{1 - s/(2a)}$ with geometry parameter q; normalized time of flight $T(x, q)$	Monotonic single-revolution $T(x, q)$ solution; derivative dT/dx supplied	—	Reconstructed from auxiliary formulas

Continued on next page

Table 1 : Continued

Reference	Iteration Variable	Iteration Algorithm	Initial Guess	Velocity Reconstruction
Bate et al. (1971)	Universal variable $z = \Delta E^2$ (ellipse) or $-\Delta F^2$ (hyperbola)	Newton–Raphson (1st order) or bisection	$z_0 = 0$ (parabolic orbit) for NR; $z \in [-4\pi^2, 4\pi^2]$ bracket for bisection	Gauss f and g functions; singular at $\theta = \pi$ ($g = 0$)
Simó (1973)	$z = (\Delta E)^2/4$; universal via Stumpff functions	Modified Newton–Raphson with boundary safety checks (MNR)	Interpolating table (4D: z, θ, r_2/r_1, t); or $z_0 = (\theta/2)^2$ for ellipse, $z_0 = 0$ for hyperbola	Orbital elements
Sun (1979)	Time/angular travel (multi-revolution)	Laguerre (modified)	Graphical bracketing	—
Battin and Vaughan (1984)	Sector area-to-triangle area ratio y (continued fraction form)	Successive substitution (direct cubic root)	x (specific) for elliptical orbits; $x = 0$ otherwise	Orbital elements
Gooding (1990)	x from Lancaster and Blanchard ($x^2 = 1 - s/2a$)	Halley (2nd order Householder)	Bilinear approximation for $x < 0$; weighted combination for $x > 0$; semi-empirical multi-revolution starters	Radial and transversal components
Avanzini (2008)	Transverse eccentricity $e_\perp$ (transformed x)	Newton–Raphson with numerical derivatives	$e_T = 0$ (fundamental ellipse; $x_0 = 0$)	Orbital elements
He (2010)	Transverse eccentricity e_T (transformed x)	Newton–Raphson (analytical derivative)	$N = 0$: $x_0 = 0$; $N \geq 1$: previous root ± 1	Orbital elements

Continued on next page

Table 1 : Continued

Reference	Iteration Variable	Iteration Algorithm	Initial Guess	Velocity Reconstruction
Vallado (2013)	Same z as Bate	Bisection (guaranteed convergence)	(asymptote); lower bound $-4\pi^2$ (extended for high-eccentricity orbits)	Gauss f and g functions
Arora and Russell (2013)	Cosine transformation of Bate's universal variable	High-order root solver	Sophisticated initial guess (fewer inverse trig functions)	—
Izzo (2015)	x from Lancaster and Blanchard; near-parabolic: switches to z	Householder (3rd order); also Regula–Falsi in ξ-τ plane	Piecewise linear approximation in transformed ξ-τ variables; constant starters for RF version	Radial and transversal components
Russell (2019)	Vercosine formulation; 2D parameter-space interpolation	—	Pre-computed interpolation tables ($\sim$1 MB total for all M)	Up to 3rd-order convergence
McElreath et al. (2025)	$x = \sqrt{\alpha}\chi = \Delta E$ (ellipse) or $j\Delta H$ (hyperbola)	Halley (minimum time) / Householder (3rd order)	Piecewise approximations based on transfer angle and parabolic time	Stark transformation (chord/radial components)

Table 2 below surveys the available MATLAB implementations of the main Lambert problem solvers discussed in the literature.

Table 2: Available MATLAB implementations of Lambert problem solvers.

Original algorithm	**MATLAB implementation / source**	**Key features / limitations**
Bate (1971)	"lambert_universal" by Shiva Iyer	Universal variable formulation; elliptic, parabolic, hyperbolic; no multi-revolution.
Battin (1984)	"lambert_battin" by David Eagle	Ported from FORTRAN; single revolution ($<2\pi$); prograde/retrograde.
Gooding (1990)	"lambert_gooding" by David Eagle	Ported from FORTRAN; single revolution ($<2\pi$); prograde/retrograde.
Sun (1979)	"lambert_sun" by David Eagle	Ported from FORTRAN (Wagner); works for transfers <1 revolution; prograde/retrograde.
Izzo (2015)	"Robust solver for Lambert's problem" by Rody Oldenhuis	Hybrid (Izzo fast + Gooding robust); short/long way; multi-revolution (m<10 typically); all conics.
McElreath et al. (2025)	"Universal multi-revolution Lambert solver" by James McElreath	Implements McElreath et al. (2025); 27–49% faster than Gooding; handles all conics and multi-revolution.

1.3 Objectives

The project develops a complete MATLAB framework with three solvers:

- **Solver 1**: classical two-body Lambert problem, which for a specified transfer time (Δt), determines the initial and final velocities:

$$\left[\mathbf{v}_1^{(\mathrm{tr})}, \mathbf{v}_2^{(\mathrm{tr})}\right] = \mathrm{Lambert}(\mathbf{r}_1, \mathbf{r}_2, \Delta t) \tag{8}$$

- **Solver 2**: minimization of $\Delta V_{\mathrm{total}}$ over a feasible range of transfer times $[\Delta t_{\min}, \Delta t_{\max}]$ to determine the optimal transfer time Δt_{opt}:

$$\left[\mathbf{v}_1^{(\mathrm{tr})}, \mathbf{v}_2^{(\mathrm{tr})}\right] = \mathrm{Lambert}(\mathbf{r}_1, \mathbf{r}_2, \Delta t_{\min}, \Delta t_{\max}) \tag{9}$$

$$\Delta t_{\mathrm{opt}} = \underset{\Delta t \in [\Delta t_{\min}, \Delta t_{\max}]}{\arg\min} \Delta V_{\mathrm{total}}(\Delta t). \tag{10}$$

- **Solver 3**: high-precision solution incorporating the J_2 gravitational perturbation that accounts for secular variations in the orbital elements caused by Earth's oblateness.

$$\left[\mathbf{v}_1^{(\mathrm{tr})}, \mathbf{v}_2^{(\mathrm{tr})}\right] = \mathrm{Lambert}(\mathbf{r}_1, \mathbf{r}_2, \Delta t, J_2) \tag{11}$$

The output is a set of reusable MATLAB codes, and an analysis report focusing on program correctness, code readability, and error control.

2 Methodology

This section presents a systematic technical workflow for developing a MATLAB-based numerical framework to solve Lambert transfer problem. The methodology integrates the theoretical formulation, the classical two-body Lambert solution, the optimization of transfer time for minimum velocity increment, and the incorporation of J_2 perturbation effects through numerical correction techniques. It is structured into three solvers: Solver 1 computes the classical two-body solution, Solver 2 determines the optimal transfer time minimizing total Δv, and Solver 3 refines the solution by incorporating J_2 effects.

2.1 Solver 1: Solving a Typical Lambert Transfer Problem

Section source: (Hu, 2015)

Solver 1 treats the short-way, single-revolution Lambert transfer between two circular endpoint orbits under the two-body model. The transfer conic is assumed elliptical, the transfer angle satisfies $0 < \Delta\theta < \pi$, and perturbations are neglected. Given $\mathbf{r}_1$, $\mathbf{r}_2$, Δt, and μ, the solver determines the transfer semi-major axis a and the endpoint velocity increments $\Delta\mathbf{v}_1$ and $\Delta\mathbf{v}_2$.

2.1.1 Procedural Workflow

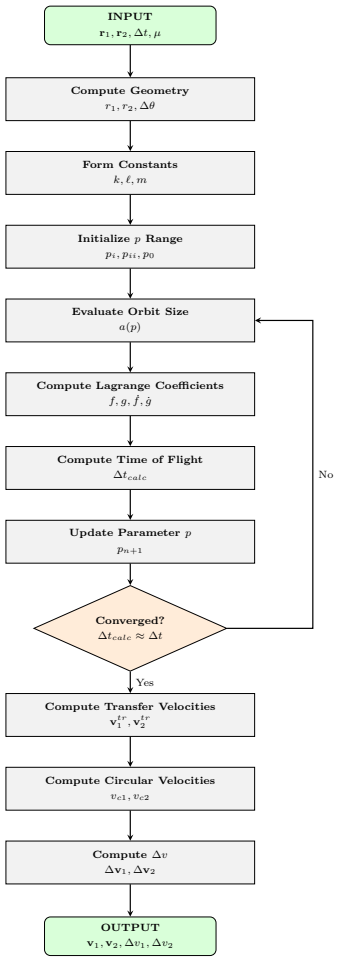


Figure 2: Solver 1 Two-Body Lambert Problem Workflow.

2.1.2 Solution for a Given Transfer Time

Geometric Quantities
Geometric quantities including the magnitudes of the position vectors, transfer angle, chord length, and semi-perimeter are derived using vector algebra and orbital geometry:

$$r_1 = \sqrt{\mathbf{r}_1 \cdot \mathbf{r}_1} \qquad r_2 = \sqrt{\mathbf{r}_2 \cdot \mathbf{r}_2} \tag{12}$$

$$\Delta\theta = \cos^{-1}\left(\frac{\mathbf{r}_1 \cdot \mathbf{r}_2}{r_1 r_2}\right) \tag{13}$$

Iteration Constants (p-Formulation)
Define the geometric constants used in the p-iteration method:

$$k = r_1 r_2(1 - \cos\Delta\theta), \quad \ell = r_1 + r_2, \quad m = r_1 r_2(1 + \cos\Delta\theta) \tag{14}$$

Bounds and Initial Guess for p
The semi-latus rectum p is the iteration variable. Determine the limiting values of p corresponding to the parabolic cases (pi and pii) to ensure the iteration stays within valid bounds:

$$p_i = \frac{k}{\ell + \sqrt{2m}}, \qquad p_{ii} = \frac{k}{\ell - \sqrt{2m}} \tag{15}$$

These bounds are not arbitrary — they are derived from the condition that the transfer orbit must be a valid ellipse connecting $\vec{r}_1$ and $\vec{r}_2$. Any value of p outside $(p_i,\ p_{ii})$ does not correspond to a real elliptic transfer. The initial guess is set to the geometric mean of the two limiting values:

```
p = sqrt(p_i * p_ii);
```

This ensures the first iteration starts inside the valid domain without biasing the search toward either parabolic limit. The secondary p-step tolerance is set to 10^{-10} m, while the main convergence criterion is the time-of-flight residual discussed below.

This ensures the first iteration starts inside the valid domain. The iteration tolerance p is set to 10^{-10} meters, which is sufficient for the precision required in this study.

Newton-Raphson Iteration
The maximum iteration count is set to 500. This limit is chosen conservatively — no analytical bound on the number of iterations exists a priori, so a generous ceiling is imposed to guarantee termination. In practice, convergence is reached in far fewer steps for well-conditioned cases.

At each iteration, the following quantities are computed in sequence, all as explicit functions of the current p:

Semi-major axis (from p):

$$a = \frac{mkp}{(2m - l^2)p^2 + 2klp - k^2} \tag{16}$$

Lagrange coefficients (from p and a):

$$f = 1 - \frac{r_2}{p}(1 - \cos\Delta\theta), \quad g = \frac{r_1 r_2 \sin\Delta\theta}{\sqrt{\mu p}}, \quad \dot{g} = 1 - \frac{r_1}{p}(1 - \cos\Delta\theta) \tag{17}$$

Change in eccentric anomaly (from f and a):

The eccentric anomaly difference ΔE is obtained from the Lagrange coefficient f using the relation:

$$\cos(\Delta E) = 1 - \frac{r_1}{a}(1 - f) \tag{18}$$

Due to floating-point errors during the iterative solution, this quantity may slightly exceed the valid range $[-1, 1]$. Therefore, a numerical clamp is applied:

```
cosDE = max(-1, min(1, 1 - (r1/a)*(1-f)));
```

The reason is that during early iterations, the parameters a and f are not yet converged. As a result, numerical inaccuracies may push the argument of the inverse trigonometric function outside its domain. Without this safeguard, MATLAB would return `NaN`, which would propagate through subsequent calculations and prevent convergence of the iterative scheme. The clamp enforces the theoretical bounds of the cosine function and ensures numerical robustness.

To avoid the ambiguity associated with the arccos function and to ensure correct quadrant selection, the eccentric anomaly is instead computed using the two-argument arctangent function:

$$\sin(\Delta E) = \sqrt{1 - \cos^2(\Delta E)} \tag{19}$$

For the short-way transfer case ($0 < \Delta\theta < \pi$), $\sin(\Delta E)$ is strictly positive. The eccentric anomaly is then obtained as:

$$\Delta E = \text{atan2}(\sin(\Delta E), \cos(\Delta E)) \tag{20}$$

The use of `atan2` ensures an unambiguous determination of ΔE by correctly identifying the angle quadrant, unlike arccos, which returns values only in $[0, \pi]$. This approach improves numerical stability and guarantees consistency throughout the iterative Lambert solution.

Calculated time of flight (from g, a, ΔE):

$$\Delta t_{\text{calc}} = g + \sqrt{\frac{a^3}{\mu}}(\Delta E - \sin\Delta E) \tag{21}$$

Update rule — the derivative $\frac{dt}{dp}$ is formed from the Lagrange coefficients, a, and ΔE, all of which depend only on the current p. The Newton-Raphson step is therefore:

$$p_{n+1} = p_n + \frac{\Delta t - \Delta t_{\text{calc}}}{\frac{dt}{dp}\Big|_{p_n}} \tag{22}$$

where dt/dp is:

$$\frac{dt}{dp} = -\frac{g}{2p} - \frac{3}{2}a(\Delta t_{calc} - g)\left(\frac{k^2 + (2m - \ell^2)p^2}{mkp^2}\right) + \sqrt{\frac{a^3}{\mu}}\frac{2k\sin\Delta E}{p(k - \ell p)} \tag{23}$$

The direct evaluation of (23) may suffer from numerical instabilities, particularly when the denominator $(k - \ell p)$ approaches zero.

Numerical Safeguards

Two main issues are addressed:

- **Near-singularity in $(k - \ell p)$:** When $k - \ell p \to 0$, the last term becomes unbounded. A lower bound is imposed to prevent division by very small values.
- **Vanishing derivative:** If $\frac{dt}{dp} \approx 0$, the Newton update becomes unstable (division by a very small number). In this case, a fallback bisection-like step is used instead.

```
p = (p + sqrt(p_i*p_ii)) / 2;
```

These safeguards ensure that the iterative scheme remains stable even in regions where the analytical expression becomes ill-conditioned. In particular, they prevent numerical divergence during early iterations and near singular configurations of the Lambert solution.

After each update, p is clamped to remain inside the physical domain:

```
p = max(p_i * 1.0001, min(p_ii * 0.9999, p));
```

The iteration terminates when the residual satisfies:

$$|\Delta t - \Delta t_{\text{calc}}| < \varepsilon = 10^{-3}\ \text{s} \tag{24}$$

At that point the Lagrange coefficients f, g, $\dot{g}$ have converged to the values consistent with the desired transfer time.

Transfer Orbit Velocities

With the converged Lagrange coefficients, the two transfer velocities follow directly:

$$\vec{v}_1^{(\text{tr})} = \frac{\vec{r}_2 - f\,\vec{r}_1}{g}, \qquad \vec{v}_2^{(\text{tr})} = \frac{\dot{g}\,\vec{r}_2 - \vec{r}_1}{g} \tag{25}$$

Circular Orbit Velocity Vector Direction

For a circular orbit, the velocity vector must satisfy two fundamental conditions: it must be perpendicular to the position vector and lie within the orbital plane. The magnitude of the circular velocity is given by:

$$v_c = \sqrt{\frac{\mu}{r}} \tag{26}$$

while its direction is determined from the orbital geometry.

Velocity Direction Construction:

The orbital plane is defined by the initial and final position vectors. A normal vector to this plane is obtained using the cross product:

$$\mathbf{h} = \mathbf{r}_1 \times \mathbf{r}_2 \tag{27}$$

The circular velocity direction is constructed such that it is orthogonal to the radial direction and lies in the orbital plane. This is achieved using the vector relation:

$$\hat{\mathbf{v}}_c = \hat{\mathbf{h}} \times \hat{\mathbf{r}} \tag{28}$$

Thus, the circular velocity vectors at the initial and final positions are given by:

$$\mathbf{v}_{c1} = \sqrt{\frac{\mu}{r_1}} \left(\hat{\mathbf{h}} \times \hat{\mathbf{r}}_1 \right), \qquad \mathbf{v}_{c2} = \sqrt{\frac{\mu}{r_2}} \left(\hat{\mathbf{h}} \times \hat{\mathbf{r}}_2 \right) \tag{29}$$

```
plane_normal = cross(r1_vec, r2_vec);
plane_normal_uv = plane_normal / norm(plane_normal);

vc1_vec = sqrt(mu / r1) * cross(plane_normal_uv, r1_vec / r1);
vc2_vec = sqrt(mu / r2) * cross(plane_normal_uv, r2_vec / r2);
```

The required velocity increments to perform the transfer are then:

$$\Delta v_1 = \left| \vec{v}_1^{(\mathrm{tr})} - \vec{v}_1^{(\mathrm{circ})} \right|, \quad \Delta v_2 = \left| \vec{v}_2^{(\mathrm{tr})} - \vec{v}_2^{(\mathrm{circ})} \right| \tag{30}$$

$$\Delta v_{\mathrm{total}} = \Delta v_1 + \Delta v_2 \tag{31}$$

These norms measure the magnitude of the maneuver required at each endpoint — the actual fuel cost of the transfer.

From this sequence, the core function is defined:

$$[\mathbf{v}_1^{(\mathrm{tr})}, \mathbf{v}_2^{(\mathrm{tr})}] = \mathrm{Lambert}(\mathbf{r}_1, \mathbf{r}_2, \Delta t) \tag{32}$$

This procedure yields a numerically stable and efficient solution for the Lambert transfer problem using the p-iteration method.

2.2 Solver 2: Minimisation of Total Velocity Increment

Solver 2 extends the single-transfer-time computation of Solver 1 to a scalar optimisation problem. Given the position vectors $\mathbf{r}_1$ and $\mathbf{r}_2$, the transfer time Δt optimal is treated as a free variable and the objective is to find the value Δt optimal that minimises the total propellant expenditure:

$$C(\Delta t_{opt}) = \Delta v_{\mathrm{total}}(\Delta t_{opt}) = \Delta v_1 + \Delta v_2 \tag{33}$$

2.2.1 Procedural Workflow

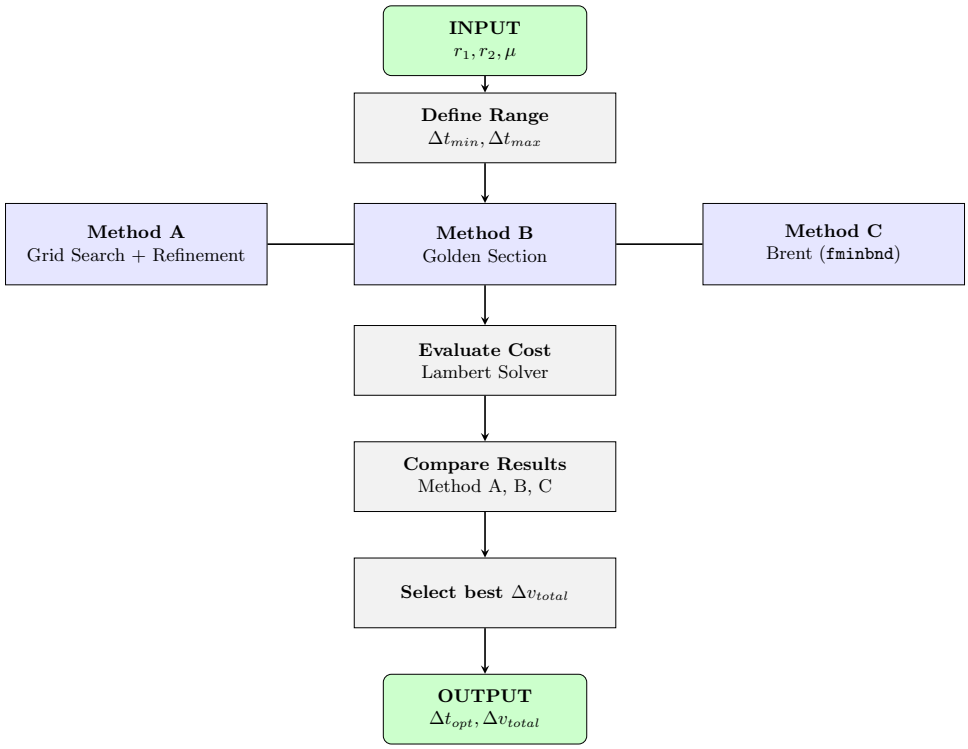


Figure 3: Solver 2 Fuel Transfer Optimization Workflow.

2.2.2 Transfer-Time Bounds

A physically meaningful search interval $[\Delta t_{\min}, \Delta t_{\max}]$ must be established before any optimization is performed.

Lower bound — Hohmann time:
The Hohmann transfer connects two coplanar circular orbits with a half-ellipse. Its transfer time equals the time to traverse half the ellipse (Hohmann, 1925):

$$\Delta t_H = \pi \sqrt{\frac{(r_1 + r_2)^3}{8\mu}} \tag{34}$$

Although the Hohmann transfer time is commonly taken as a baseline reference for bounding feasible transfer durations, this benchmark only corresponds to the minimum-energy trajectory rather than the true theoretical minimum flight time. A relaxation factor of 0.1 is therefore adopted to permit high-speed high-energy transfers, while ruling out unreasonably short flight times that entail infeasibly large velocities and deteriorate numerical stability.

Upper bound — mean-orbit period:
The upper bound is set to one full orbital period of the mean orbit whose semi-major axis is $a_m = (r_1 + r_2)/2$.
This bound encloses the fuel-optimal transfer time for all single-revolution elliptic ge-

ometries and transfer times exceeding this value would correspond to multi-revolution solutions, which are outside the scope of the present study.

```
dt_min = 0.1 * pi * sqrt((r1 + r2)^3 / (8 * mu));
dt_max = 2 * pi * sqrt(((r1 + r2) / 2)^3 / mu);
```

2.2.3 Cost Function

For each candidate Δt_i in the search interval, the Lambert solver (Solver 1) is called to obtain the transfer velocities, and the total Δv is evaluated:

$$C(\Delta t_i) = \left\| \mathbf{v}_1^{(\mathrm{tr})}(\Delta t_i) - \mathbf{v}_{c1} \right\| + \left\| \mathbf{v}_2^{(\mathrm{tr})}(\Delta t_i) - \mathbf{v}_{c2} \right\| \tag{35}$$

If the Lambert solver fails to converge at a particular Δt_i (e.g. at near-degenerate geometries close to the bounds), the value is excluded from the minimum search.

2.2.4 Method A: Grid Search with Local Refinement

Grid search is the simplest deterministic global optimisation strategy (Nocedal and Wright, 2006). The search interval is uniformly discretised into $N_{\mathrm{grid}} = 500$ candidate points. The cost function is evaluated at each point, and the coarse global minimiser is identified:

$$\Delta t_{\mathrm{coarse}} = \arg\min_i C(\Delta t_i), \quad \Delta t_i \in \left[\Delta t_{\min}, \Delta t_{\max}\right], \quad i = 1, \ldots, N_{\mathrm{grid}} \tag{36}$$

The theoretical justification for the grid-search approach is the unimodality of the cost function within the single-revolution domain: Vallado (2013) shows that the total Δv cost exhibits a single global minimum inside the physically feasible interval.
A fine grid of $N_{\mathrm{fine}} = 200$ points is then constructed inside a window of ± 3 coarse grid cells around $\Delta t_{\mathrm{coarse}}$.
The total cost is $N_{\mathrm{grid}} + N_{\mathrm{fine}} = 700$ Lambert solver calls.

```
window = (dt_max - dt_min) / n_grid * 3;
dt_lo = max(dt_min * 1.01, dt_grid_opt - window);
dt_hi = min(dt_max * 0.99, dt_grid_opt + window);
n_fine = 200;
```

2.2.5 Method B: Golden Section Search

The golden section search is a derivative-free interval-reduction method formalised by Kiefer (1953). It is the optimal strategy, in the minimax sense, for locating the minimum of a unimodal scalar function using the fewest evaluations, given only bracketing information.

The algorithm exploits the golden ratio $\varphi = (\sqrt{5} - 1)/2 \approx 0.6180$ to place interior evaluation points such that one point can always be reused in the next iteration

```
c = b - \varphi * (b - a);
d = a + \varphi * (b - a);
```

At each step the bracket shrinks by a factor of φ: if $C(c) < C(d)$ the right portion $[d, b]$ is discarded and $d \leftarrow c$; otherwise the left portion $[a, c]$ is discarded and $c \leftarrow d$. After n steps

starting from an interval of length L, the bracket has been reduced to $\varphi^n L$ (Kiefer, 1953).

The iterative process continues until the length of the bracketing interval becomes sufficiently small:

$$(b - a) \leq \text{tol} \tag{37}$$

where `tol` is a user-defined tolerance that controls the accuracy of the solution.

At this stage, the interval $[a, b]$ is considered to contain the minimizer with acceptable precision. Instead of selecting one of the interior points (c or d), the optimal solution is approximated by the midpoint of the final interval.

```
tol = 1e-4;
x_opt = (a + b) / 2;
f_opt = f(x_opt);
```

2.2.6 Method C: Brent's Method (`fminbnd`)

Brent's method is a hybrid scalar minimisation algorithm that combines the global bracketing guarantee of golden-section search with the super-linear convergence of parabolic interpolation (Brent, 1973). Starting from the interval $[\Delta t_{\min}, \Delta t_{\max}]$, the algorithm maintains three points x, w, v (current best, second-best, and most-recently-visited). At each step it fits a parabola through these three points and evaluates its minimum; if the parabolic step lies within the bracket and does not exceed half the previous step size, it is accepted; otherwise the algorithm reverts to a golden-section reduction (Brent, 1973).

For smooth unimodal functions, as this present study, the method converges super-linearly; the golden-section fallback guarantees linear convergence at rate $\varphi \approx 0.618$ per step in the worst case (Kiefer, 1953).

MATLAB's built-in `fminbnd` is a direct implementation of this algorithm.

```
options_C = optimset('TolX', 1e-6, 'MaxFunEvals', 500, 'Display', 'off');
[dt_C, dv_C, exitflag_C, output_C] = fminbnd(safe_cost, dt_min * 1.01, dt_max
* 0.99, options_C);
```

2.3 Solver 3: Lambert Transfer with J_2 Perturbation

Solver 3 extends the two-body Lambert solution into a perturbed boundary-value problem. The objective is no longer only to find the Keplerian conic that joins $\mathbf{r}_1$ and $\mathbf{r}_2$ in a prescribed time, but to find an initial velocity $\mathbf{v}_{1,J_2}$ such that the numerically propagated trajectory, including the Earth's oblateness term J_2, reaches the same terminal position at the same transfer time. The method implemented in MATLAB follows the standard two-step perturbed Lambert strategy:

1. compute a two-body Lambert seed from the Lagrange f-g state transition relations;
2. correct the initial velocity by shooting, using the state transition matrix (STM) of the perturbed dynamics.

2.3.1 Procedural Workflow

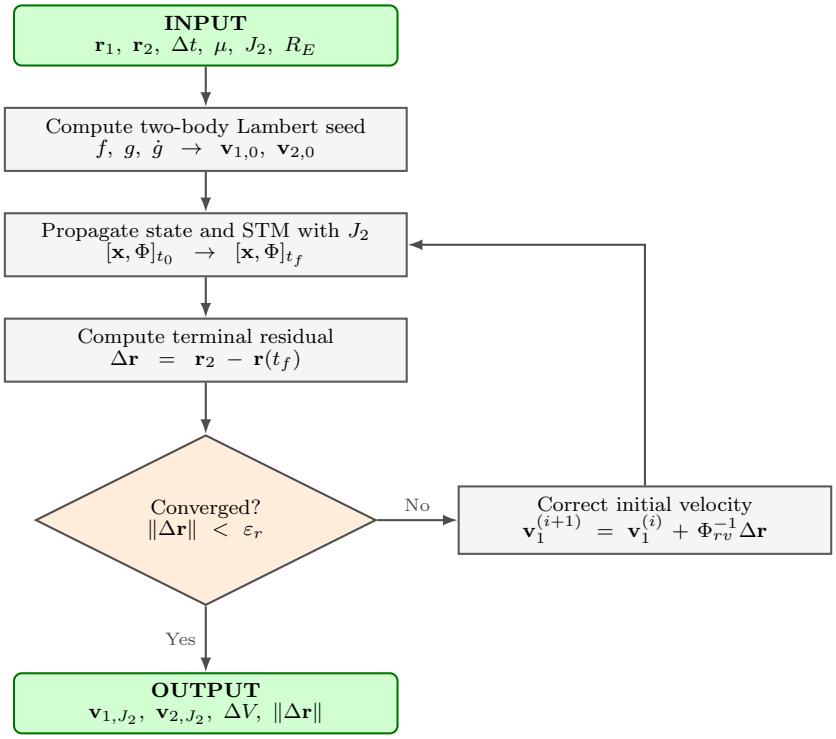


Figure 4: Solver 3 workflow for the J_2-perturbed Lambert correction.

2.3.2 Two-Body Seed

(Hu, 2015)

Solver 3 first constructs a two-body Lambert solution, which is used as the initial guess for the later J_2-perturbed shooting correction. If Solver 3 is executed alone, it uses the transfer time entered by the user. If it is executed after Solver 2, it uses the optimized transfer time Δt_{opt} returned by Solver 2:

$$\Delta t_{S3} = \begin{cases} \Delta t_{\text{input}}, & \text{Solver 3 without Solver 2,} \\ \Delta t_{\text{opt}}, & \text{Solver 3 with Solver 2.} \end{cases} \tag{38}$$

The same `lambert_solver` function used in Solver 1 is called to compute this initial two-body solution. Therefore, Solver 3 does not introduce a separate Lambert formulation; it reuses the validated p-iteration solver as the Keplerian seed generator.

For a trial value of the semi-latus rectum p, the Lagrange coefficients $f(p)$, $g(p)$, and $\dot{g}(p)$ define the two-body state transition between the endpoints. The eccentric-anomaly difference is obtained from

$$\cos\Delta E(p) = 1 - \frac{r_1}{a(p)}\left[1 - f(p)\right], \tag{39}$$

and the corresponding two-body time-of-flight function is

$$\Delta t(p) = g(p) + \sqrt{\frac{a(p)^3}{\mu}} \left[\Delta E(p) - \sin \Delta E(p)\right]. \tag{40}$$

The numerical iteration solves

$$F(p) = \Delta t_{S3} - \Delta t(p) = 0, \tag{41}$$

inside the short-way, single-revolution domain. After convergence, the code checks that the selected transfer time is shorter than the period of the resulting transfer ellipse:

$$\Delta t_{S3} < T_{\text{tr}} = 2\pi \sqrt{\frac{a_{\text{tr}}^3}{\mu}}. \tag{42}$$

Once the Lagrange coefficients have converged, the MATLAB implementation reconstructs the two-body endpoint velocities as

$$\mathbf{v}_{1,0} = \frac{\mathbf{r}_2 - f\mathbf{r}_1}{g}, \qquad \mathbf{v}_{2,0} = \frac{\dot{g}\,\mathbf{r}_2 - \mathbf{r}_1}{g}. \tag{43}$$

The subscript 0 indicates that these velocities are only the two-body initial values. They satisfy the Keplerian Lambert problem, but they are not yet corrected for the J_2 perturbation.

2.3.3 Perturbed Equations of Motion

(Hu, 2015)

After the two-body seed has been obtained, Solver 3 replaces the Keplerian propagation with an oblate-Earth gravity model. The propagated state vector is

$$\mathbf{x}(t) = \begin{bmatrix} \mathbf{r}(t) \\ \mathbf{v}(t) \end{bmatrix}, \qquad \dot{\mathbf{x}}(t) = \begin{bmatrix} \mathbf{v}(t) \\ \mathbf{a}(\mathbf{r}) \end{bmatrix}. \tag{44}$$

Here $\mathbf{a}(\mathbf{r})$ contains both the central two-body acceleration and the first-order J_2 correction. For an axisymmetric nonspherical Earth, the specific gravitational potential energy may be expanded in zonal harmonics as

$$U(r, \delta) = -\frac{\mu}{r}\left[1 - \sum_{n=2}^{\infty} J_n \left(\frac{R_E}{r}\right)^n P_n(\sin \delta)\right], \tag{45}$$

where J_n are the zonal-harmonic coefficients, P_n are the Legendre polynomials, and δ is the geocentric latitude. Retaining only the dominant nonspherical term, J_2, with its contribution

$$U_2 = \frac{\mu J_2 R_E^2}{r^3} P_2(\sin \delta). \tag{46}$$

Given

$$\sin\delta = \frac{z}{r}, \qquad P_2(\sin\delta) = \frac{1}{2}\left(3\sin^2\delta - 1\right), \tag{47}$$

substitution into Equation 46 gives

$$U_2 = \frac{\mu J_2 R_E^2}{r^3}\left(-\frac{1}{2} + \frac{3}{2}\sin^2\delta\right) = \frac{\mu J_2 R_E^2}{2}\left(-r^{-3} + 3z^2 r^{-5}\right). \tag{48}$$

Let $C = \mu J_2 R_E^2$ and $r = \sqrt{x^2+y^2+z^2}$. Using $\partial r^{-n}/\partial x_i = -n x_i r^{-n-2}$, differentiation of Equation 48 gives

$$\begin{aligned}
\frac{\partial U_2}{\partial x} &= \frac{3Cx}{2r^5}\left(1 - \frac{5z^2}{r^2}\right), \\
\frac{\partial U_2}{\partial y} &= \frac{3Cy}{2r^5}\left(1 - \frac{5z^2}{r^2}\right), \\
\frac{\partial U_2}{\partial z} &= \frac{3Cz}{2r^5}\left(3 - \frac{5z^2}{r^2}\right).
\end{aligned} \tag{49}$$

The complete specific potential energy is $U = -\mu/r + U_2$, and the acceleration is $\mathbf{a} = -\nabla U$. Therefore, with $\mathbf{r} = [x\ y\ z]^T$, the acceleration used in the MATLAB implementation is

$$\begin{aligned}
a_x &= x\left[-\frac{\mu}{r^3} - \frac{3J_2\mu R_E^2}{2r^5}\left(1 - \frac{5z^2}{r^2}\right)\right], \\
a_y &= y\left[-\frac{\mu}{r^3} - \frac{3J_2\mu R_E^2}{2r^5}\left(1 - \frac{5z^2}{r^2}\right)\right], \\
a_z &= z\left[-\frac{\mu}{r^3} - \frac{3J_2\mu R_E^2}{2r^5}\left(3 - \frac{5z^2}{r^2}\right)\right].
\end{aligned} \tag{50}$$

This model is used for every propagation inside the shooting loop. Therefore, the final corrected trajectory is not a two-body Lambert arc; it is the trajectory generated by the J_2-perturbed equations of motion from the corrected initial velocity.

2.3.4 State Transition Matrix Propagation

(Battin, 1999)

The shooting correction requires the sensitivity of the final position to the initial velocity. This sensitivity is obtained by propagating the State Transition Matrix (STM) together with the spacecraft state. The STM maps a small perturbation in the initial state to the corresponding first-order perturbation in the final state:

$$\delta\mathbf{x}(t_f) = \Phi(t_f, t_0)\,\delta\mathbf{x}(t_0), \qquad \Phi(t_0, t_0) = I_6. \tag{51}$$

For the Cartesian state $\mathbf{x} = [\mathbf{r}^T \ \mathbf{v}^T]^T$, the 6×6 STM is partitioned into four 3×3 sensitivity blocks:

$$\Phi(t,t_0) = \frac{\partial \mathbf{x}(t)}{\partial \mathbf{x}(t_0)} = \begin{bmatrix} \dfrac{\partial \mathbf{r}(t)}{\partial \mathbf{r}(t_0)} & \dfrac{\partial \mathbf{r}(t)}{\partial \mathbf{v}(t_0)} \\ \dfrac{\partial \mathbf{v}(t)}{\partial \mathbf{r}(t_0)} & \dfrac{\partial \mathbf{v}(t)}{\partial \mathbf{v}(t_0)} \end{bmatrix} = \begin{bmatrix} \Phi_{rr} & \Phi_{rv} \\ \Phi_{vr} & \Phi_{vv} \end{bmatrix}. \tag{52}$$

The block Φ_{rr} gives the final-position sensitivity to the initial position, Φ_{rv} gives the final-position sensitivity to the initial velocity, and the lower blocks provide the corresponding final-velocity sensitivities. Since Solver 3 keeps $\mathbf{r}(t_0)$ fixed and corrects only $\mathbf{v}(t_0)$, the shooting update uses the block Φ_{rv}.

The STM satisfies the variational equation

$$\dot{\Phi}(t,t_0) = A(t)\Phi(t,t_0), \tag{53}$$

where the Jacobian of the perturbed dynamics has the block form

$$A(t) = \frac{\partial \dot{\mathbf{x}}}{\partial \mathbf{x}} = \begin{bmatrix} 0_{3\times3} & I_{3\times3} \\ \dfrac{\partial \mathbf{a}}{\partial \mathbf{r}} & 0_{3\times3} \end{bmatrix}. \tag{54}$$

The physical state and STM are integrated as one augmented vector,

$$\mathbf{y}_{\text{aug}}(t) = \begin{bmatrix} \mathbf{x}(t) \\ \text{vec}\left(\Phi(t,t_0)\right) \end{bmatrix} \in \mathbb{R}^{42}. \tag{55}$$

The augmented formulation ensures that the STM is evaluated along the same perturbed trajectory used to compute the terminal residual. This is important because the shooting update depends on the local dynamics of the corrected J_2 trajectory, not on the original two-body Lambert conic.

2.3.5 RK4 Propagation of the Augmented State

Solver 3 propagates the 42-dimensional augmented system using the classical fourth-order Runge–Kutta method developed by Runge (1895) and Kutta (1901). RK4 is a single-step method: it advances the solution using four evaluations of the augmented equations of motion within the current interval and does not require stored values from earlier steps. A nominal fixed step size of $S = 10$ s is used. If the remaining propagation time is smaller than S, the final step is shortened so that the integration terminates exactly at the selected transfer time.

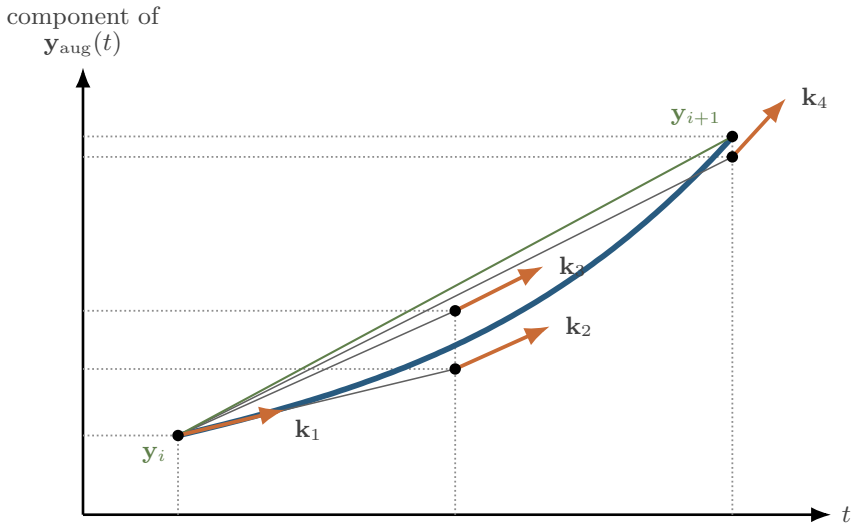


Figure 5: Geometric interpretation of one RK4 step applied to a component of Solver 3.

For a general augmented system

$$\dot{\mathbf{y}}_{\text{aug}} = \mathbf{F}(t, \mathbf{y}_{\text{aug}}), \tag{56}$$

one RK4 step from t_i to $t_i + h$ is

$$\mathbf{k}_1 = \mathbf{F}(t_i, \mathbf{y}_i), \tag{57}$$

$$\mathbf{k}_2 = \mathbf{F}\left(t_i + \frac{h}{2}, \mathbf{y}_i + \frac{h}{2}\mathbf{k}_1\right), \tag{58}$$

$$\mathbf{k}_3 = \mathbf{F}\left(t_i + \frac{h}{2}, \mathbf{y}_i + \frac{h}{2}\mathbf{k}_2\right), \tag{59}$$

$$\mathbf{k}_4 = \mathbf{F}\left(t_i + h, \mathbf{y}_i + h\mathbf{k}_3\right). \tag{60}$$

The propagated value is then obtained from the weighted average

$$\mathbf{y}_{i+1} = \mathbf{y}_i + \frac{h}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right). \tag{61}$$

In Figure 5, the blue curve illustrates one scalar component of $\mathbf{y}_{\text{aug}}$. In the code, the same update is applied to all 42 components, so the spacecraft state and the STM remain synchronized during propagation. The four derivative evaluations form a weighted estimate of the rate over the interval. The classical RK4 formula has local truncation error of order $\mathcal{O}(h^5)$ and accumulated global error of order $\mathcal{O}(h^4)$ Vallado (2013).

2.3.6 Shooting Velocity Correction

(Battin, 1999)

The two-body departure velocity $\mathbf{v}_{1,0}$ is used as the first shooting guess:

$$\mathbf{x}_0^{(0)} = \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{v}_{1,0} \end{bmatrix}. \tag{62}$$

After propagation under the J_2 dynamics, the terminal-position residual at iteration i is

$$\Delta\mathbf{r}^{(i)} = \mathbf{r}_2 - \mathbf{r}^{(i)}(t_f). \tag{63}$$

The shooting iteration is stopped when this residual satisfies

$$\left\|\Delta\mathbf{r}^{(i)}\right\| < 10^{-7}\ \text{m}. \tag{64}$$

Only the initial velocity is corrected; the initial position and the transfer time remain fixed. The relevant block of the STM is therefore

$$\Phi_{rv}(t_f, t_0) = \frac{\partial\mathbf{r}(t_f)}{\partial\mathbf{v}(t_0)} = \Phi(1{:}3, 4{:}6). \tag{65}$$

Linearizing the terminal position with respect to the initial velocity gives

$$\Delta\mathbf{r}^{(i)} \approx \Phi_{rv}^{(i)}\Delta\mathbf{v}_1^{(i)}. \tag{66}$$

Thus, each shooting iteration solves the linear system

$$\Delta\mathbf{v}_1^{(i)} = \left(\Phi_{rv}^{(i)}\right)^{-1}\Delta\mathbf{r}^{(i)}. \tag{67}$$

In the MATLAB implementation, the inverse is not formed explicitly. Instead, the equivalent linear system

$$\Phi_{rv}^{(i)}\Delta\mathbf{v}_1^{(i)} = \Delta\mathbf{r}^{(i)} \tag{68}$$

is solved using the backslash operator.

```
Phi_rv \ dr_vec
```

The departure velocity is then updated according to the Newton iteration

$$\mathbf{v}_1^{(i+1)} = \mathbf{v}_1^{(i)} + \Delta\mathbf{v}_1^{(i)}. \tag{69}$$

The process is repeated until the convergence criterion of Equation 64 is satisfied, or until the maximum number of shooting iterations is reached. After convergence, the corrected departure velocity is denoted $\mathbf{v}_{1,J_2}$, and the arrival velocity $\mathbf{v}_{2,J_2}$ is read directly from the propagated terminal state.

2.3.7 Numerical Variants Used for Validation

The baseline method is the fixed-step RK4 propagation with STM shooting described above. Two additional propagation methods are used to verify that the corrected solution is not an artifact of one numerical integrator: adaptive RK45 propagation and a fourth-order Adams–Bashforth–Moulton predictor–corrector method. All three variants propagate the

physical state and STM together and apply the same STM velocity correction. They therefore solve the same boundary condition,

$$\mathbf{r}(t_f; \mathbf{v}_1) = \mathbf{r}_2. \tag{70}$$

Adaptive RK45–STM propagation:
This validation variant is implemented with MATLAB's `ode45`. The embedded fourth-/fifth-order pair was developed by Dormand and Prince (1980), while its MATLAB ODE-suite implementation is described by Shampine and Reichelt (1997). The same 42-dimensional augmented state is propagated, but adaptive step-size selection replaces the fixed RK4 step.

From the same stage derivatives, the method forms approximations of different orders,

$$\mathbf{y}_{i+1}^{(5)} = \mathbf{y}_i + S_i \sum_{j=1}^{s} b_j^{(5)} \mathbf{k}_j, \qquad \mathbf{y}_{i+1}^{(4)} = \mathbf{y}_i + S_i \sum_{j=1}^{s} b_j^{(4)} \mathbf{k}_j. \tag{71}$$

Their difference provides an estimate of the local integration error,

$$\mathbf{e}_{i+1} = \mathbf{y}_{i+1}^{(5)} - \mathbf{y}_{i+1}^{(4)}. \tag{72}$$

After scaling the estimated error using the absolute and relative tolerances, let E_i denote the resulting normalized error measure. The step-size adjustment may be represented conceptually as

$$S_{\text{new}} = \eta S_i E_i^{-1/5}, \tag{73}$$

where $\eta < 1$ is a safety factor. MATLAB also bounds the permitted step-size change and rejects a trial step when its normalized error is too large. In the implementation, the solver options are

$$\begin{aligned} \text{RelTol} = 10^{-12}, \quad \text{AbsTol}_r = 10^{-6}\ \text{m}, \quad \text{AbsTol}_v = 10^{-9}\ \text{m/s}, \\ \text{AbsTol}_\Phi = 10^{-12}, \qquad S_{\max} = 10\ \text{s}. \end{aligned} \tag{74}$$

The STM-based shooting correction remains unchanged: after each propagation, the block Φ_{rv} is used to solve for the departure-velocity correction. Thus, this variant changes only the integration strategy.

Adams–Bashforth–Moulton STM propagation:
The second validation variant follows the fourth-order Adams–Bashforth–Moulton method presented by Vallado (2013). It is a fixed-step, multistep predictor–corrector method. Unlike RK4, it requires derivative values from earlier time levels and is therefore not self-starting. The implementation first generates three equally spaced back values using RK4.

Let

$$\mathbf{F}_n = \mathbf{F}(t_n, \mathbf{y}_n) \tag{75}$$

denote the derivative of the augmented state. The fourth-order Adams–Bashforth predictor is

$$\mathbf{y}_{n+1}^{p} = \mathbf{y}_n + \frac{S}{24}\left(55\mathbf{F}_n - 59\mathbf{F}_{n-1} + 37\mathbf{F}_{n-2} - 9\mathbf{F}_{n-3}\right). \tag{76}$$

The equations of motion are evaluated at the predicted state,

$$\mathbf{F}_{n+1}^{p} = \mathbf{F}(t_{n+1}, \mathbf{y}_{n+1}^{p}), \tag{77}$$

and the fourth-order Adams–Moulton corrector gives

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{S}{24}\left(9\mathbf{F}_{n+1}^{p} + 19\mathbf{F}_n - 5\mathbf{F}_{n-1} + \mathbf{F}_{n-2}\right). \tag{78}$$

Because multistep formulas require equally spaced back values, the implementation selects

$$N = \left\lceil \frac{\Delta t}{S_{\text{nom}}} \right\rceil, \qquad S_{\text{ABM}} = \frac{\Delta t}{N}, \qquad S_{\text{nom}} = 10 \text{ s}. \tag{79}$$

This gives a constant step no larger than 10 s while reaching the terminal time exactly. After the three RK4 startup steps, each Adams–Bashforth–Moulton step requires one evaluation at the predicted state and one at the corrected state. The method therefore reuses stored derivative history and can require fewer right-hand-side evaluations than RK4. The STM correction itself is unchanged, so RK4–STM, RK45–STM, and ABM4–STM differ only in how the augmented differential equations are propagated.

Each shooting method converges on the discrete trajectory produced by its own integrator. Consequently, its final shooting residual is not by itself an independent measure of continuous-trajectory accuracy. After convergence, the code cross-propagates each corrected departure velocity with strict `ode113` and `ode45` integrations of the six-state J_2 equations, using RelTol $= 2.5 \times 10^{-13}$ and $S_{\max} = 0.25$ s. The reported cross-propagated endpoint error is the larger of the two reference residuals,

$$\varepsilon_{\text{ref}} = \max\left(\left\|\mathbf{r}_2 - \mathbf{r}_{113}(t_f)\right\|, \left\|\mathbf{r}_2 - \mathbf{r}_{45}(t_f)\right\|\right), \tag{80}$$

and the difference between the two reference endpoints is also recorded. This conservative check separates shooting convergence from propagation accuracy.

3 Results

3.1 Application Scenario

Fundamental constants and input variables are specified, including the Earth's gravitational constant μ, J_2 gravitational perturbation coefficient, initial and final position vectors, and transfer time interval. These parameters form the input basis for all subsequent calculations, as shown in Table 3; Figure 6 illustrates the target orbit geometry defined by the two position vectors.

Table 3: Core parameters of the application scenario.

Parameter	Symbol	Value
Earth's gravitational constant	μ	3.986×10^{14} m^3 s^{-2}
J_2 coefficient	J_2	1.08263×10^{-3}
Initial position vector	$\mathbf{r}_1$	$[5000,\ 10000,\ 0]^T$ km
Final position vector	$\mathbf{r}_2$	$[14000,\ 5000,\ 0]^T$ km
Specified transfer time	Δt	3600 s

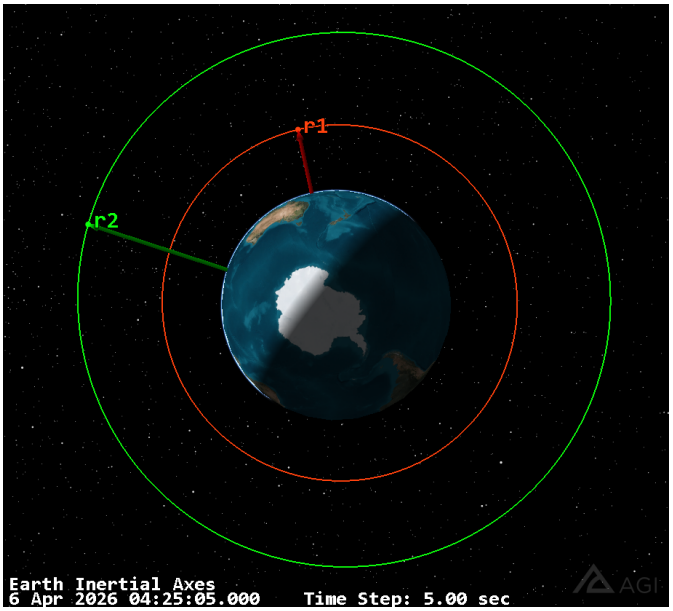


Figure 6: Orbit geometry of the application scenario defined by position vectors $\mathbf{r}_1$ and $\mathbf{r}_2$.

3.2 Input Verification

Before the numerical solvers are executed, `main_lambert.m` verifies the admissibility of the user inputs. The implemented checks are:

1. time of flight is positive when Solver 1 or Solver 3 requires a selected transfer time;
2. initial and final positions are distinct;
3. both radius magnitudes are above Earth's surface;
4. transfer angle satisfies $0 < \Delta\theta < \pi$;
5. the final solved transfer is elliptical and satisfies $\Delta t < T_{\text{tr}}$.

The following validation cases were executed using the same conditions and error messages implemented in `main_lambert.m`.

```
Scenario 1: non-positive time of flight for Solver 1
  r1 = [5000 10000 0] km
  r2 = [14000 5000 0] km
  time of flight = -3600 s
  validation result: ERROR
  Validation failed: time of flight must be positive.
```

```
Scenario 2: identical initial and final position vectors
  r1 = [5000 10000 0] km
  r2 = [5000 10000 0] km
  time of flight = 3600 s
  validation result: ERROR
  Validation failed: initial and final position vectors must be distinct.
```

```
Scenario 3: initial radius below Earth surface
  r1 = [1000 0 0] km
  r2 = [14000 5000 0] km
  time of flight = 3600 s
  validation result: ERROR
  Validation failed: initial radius must be above Earth's surface,
  greater than 6378.137 km.
```

```
Scenario 4: transfer angle outside 0 < angle < pi
  r1 = [7000 0 0] km
  r2 = [8000 0 0] km
  time of flight = 3600 s
  validation result: ERROR
  Validation failed: transfer angle must satisfy 0 < angle < pi radians.
```

3.3 Solver 1

Solver 1 was executed for the prescribed transfer time of $\Delta t = 3600$ s using the two-body p-iteration procedure described in the methodology. The terminal output for the main transfer velocities and velocity increments was:

```
=== Solver 1: Two-Body Lambert ===
v1_transfer = [+5029.824416  +1465.252592  +0.000000] m/s
v2_transfer = [-146.920623  -3121.898880  +0.000000] m/s
Delta-V1 = 4147.185520 m/s
Delta-V2 = 2577.760366 m/s
Delta-V_total = 6724.945886 m/s
```

The resulting transfer orbit was elliptical, with the following orbital-element summary:

```
Transfer orbit elements:
  Angular momentum = 42971.981203 km^2/s
  Eccentricity = 0.700191826464
  Inclination = 180.000000 deg
  True anomaly initial = 146.761743 deg
  True anomaly final = 190.542867 deg
  Semimajor axis = 9088.496978 km
  Periapse radius = 2724.805679 km
  Period = 8622.820030 s = 143.713667 min = 2.395228 hr
```

3.4 Solver 2

Solver 2 searched the transfer-time interval defined in the methodology and minimized the same total two-impulse velocity increment used in Solver 1. The computed time bounds were:

```
Transfer time bounds:
  dt_min = 739.533764 s = 0.205426 hr
  dt_max = 14790.675276 s = 4.108521 hr
```

All three scalar optimization methods converged to the same minimum within the printed numerical precision:

```
Optimization results:
  Method               dt [s]       Delta-V [m/s]   N evals      T executed
  Grid + refinement   2003.026481    3811.290954       700          0.085969
  Golden section      2002.910020    3811.290905        42          0.003400
  fminbnd / Brent     2002.909999    3811.290905        14          0.242215
  Brent exit flag = 1, where 1 means converged
  Coarse grid best before refinement: dt = 2000.060913 s,
  Delta-V = 3811.320068 m/s
```

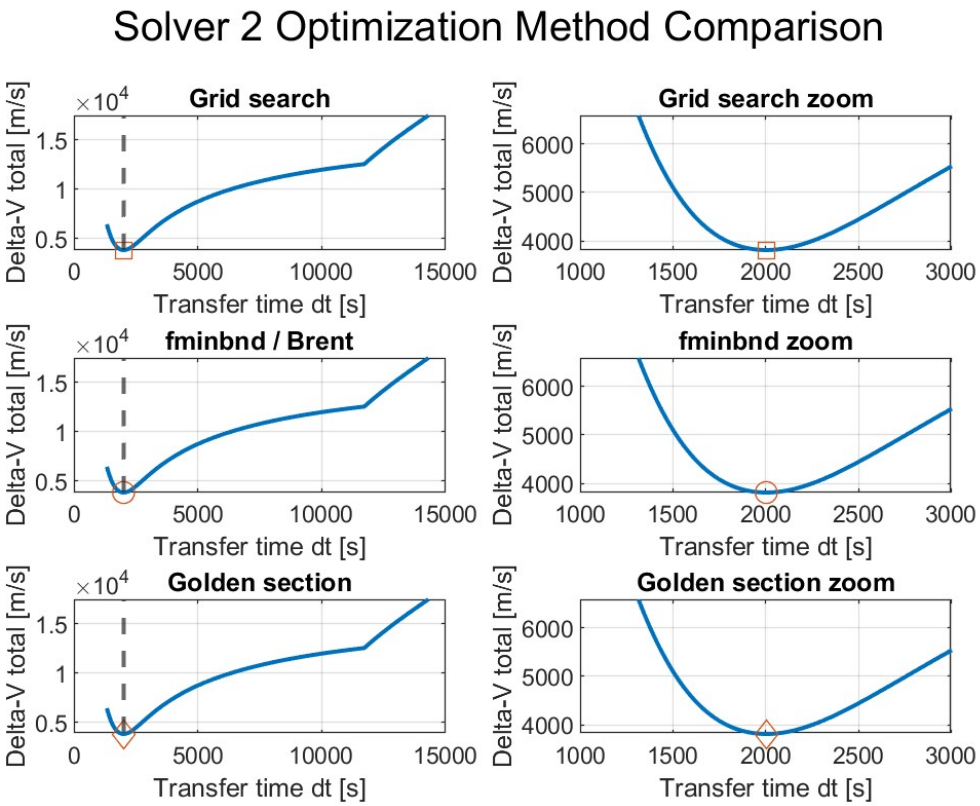


Figure 7: Plot of the three optimization methods for total ΔV as a function of TOF.

The selected optimum was the Brent result, with $\Delta t_{\text{opt}} = 2002.909999$ s and $\Delta V_{\text{total}} = 3811.290905$ m/s. The final optimized transfer state printed by the program was:

```
Optimal transfer solution using Brent (fminbnd):
  v1_transfer = [+6179.344215, -610.149783, +0.000000] m/s
  v2_transfer = [+2748.736372, -3650.036372, +0.000000] m/s
  Delta-V1 = 2224.340882 m/s
  Delta-V2 = 1586.950023 m/s
  Delta-V_total = 3811.290905 m/s
  Optimal transfer time = 2002.909999 s = 0.556364 hr
```

The corresponding optimized transfer orbit was reported as follows:

```
Transfer orbit elements:
  Angular momentum = 64844.191071 km^2/s
  Eccentricity = 0.365178191648
  Inclination = 180.000000 deg
  True anomaly initial = 98.897757 deg
  True anomaly final = 142.678881 deg
  Semimajor axis = 12172.048649 km
  Periapse radius = 7727.081935 km
  Period = 13364.623863 s = 222.743731 min = 3.712396 hr
```

3.5 Solver 3

Solver 3 was executed in two configurations. The first used the prescribed transfer time of 3600 s, matching Solver 1. The second used the optimized transfer time returned by Solver 2, $\Delta t_{\text{opt}} = 2002.909999$ s. In both cases, the two-body Lambert solution was used as the initial guess and the STM shooting method corrected the initial velocity under the J_2 perturbation model.

3.5.1 Solver 3 at the Prescribed Transfer Time

For the fixed-time case, the two-body seed was identical to Solver 1 within the displayed precision. When propagated with J_2 before correction, the terminal position residual was several kilometers. After three shooting iterations, the residual was reduced below the convergence tolerance:

```
Shooting method convergence:
  Iter  1 : ||dr|| = 6.7742188521e+03 m
  Iter  2 : ||dr|| = 7.0379476996e-01 m
  Iter  3 : ||dr|| = 4.1650011716e-09 m
  Converged in 3 iterations.

J2-corrected transfer solution:
  v1_J2 = [+5030.555406  +1466.161826  +0.000000] m/s
  v2_J2 = [-147.541981  -3122.318202  +0.000000] m/s
  Delta-V1_J2 = 4148.037506 m/s
  Delta-V2_J2 = 2577.930276 m/s
  Delta-V_total_J2 = 6725.967782 m/s
  Internal shooting residual = 4.1650011716e-09 m
  Common-reference position error = 3.6761924286e-04 m

Comparison: two-body seed vs J2-corrected transfer:
  Delta(Delta-V1) = 0.851986 m/s
  Delta(Delta-V2) = 0.169910 m/s
  Delta(Delta-V_total) = 1.021896 m/s
```

3.5.2 Solver 3 at the Solver 2 Optimized Transfer Time

When Solver 3 is called together with Solver 2, the optimized transfer time is used for the perturbed correction. This configuration keeps the lower-fuel transfer found by Solver 2 while adding the J_2 boundary correction:

```
time of flight = 2002.909999 s
case = Solver 2 optimized transfer time

Shooting method convergence:
  Iter  1 : ||dr|| = 2.4655734454e+03 m
  Iter  2 : ||dr|| = 5.3978510269e-02 m
  Iter  3 : ||dr|| = 5.6650140605e-09 m
  Converged in 3 iterations.
```

```
J2-corrected transfer solution:
  v1_J2 = [+6179.985251  -609.387419  +0.000000] m/s
  v2_J2 = [+2748.189564  -3650.417270  +0.000000] m/s
  Delta-V1_J2 = 2225.288717 m/s
  Delta-V2_J2 = 1586.308645 m/s
  Delta-V_total_J2 = 3811.597362 m/s
  Internal shooting residual = 5.6650140605e-09 m
  Common-reference position error = 7.6596235152e-05 m

Comparison: two-body seed vs J2-corrected transfer:
  Delta(Delta-V1) = 0.947836 m/s
  Delta(Delta-V2) = -0.641378 m/s
  Delta(Delta-V_total) = 0.306457 m/s
```

3.5.3 Solver 3 Numerical Method Comparison

The Solver 3 correction was checked using three propagation strategies with the same STM shooting correction: fixed-step RK4, adaptive RK45 through MATLAB's `ode45`, and fixed-step fourth-order Adams–Bashforth–Moulton propagation. All three converged in three residual evaluations. Their independently cross-propagated total ΔV values agree within 7.3×10^{-7} m/s, which is negligible at mission-design scale. The conservative endpoint errors are 7.66×10^{-5} m for RK4, 4.39×10^{-8} m for RK45, and 6.33×10^{-4} m for ABM4. ABM4 requires the fewest right-hand-side evaluations because it reuses derivative history after its RK4 startup phase, whereas the tighter adaptive RK45 validation requires the most.

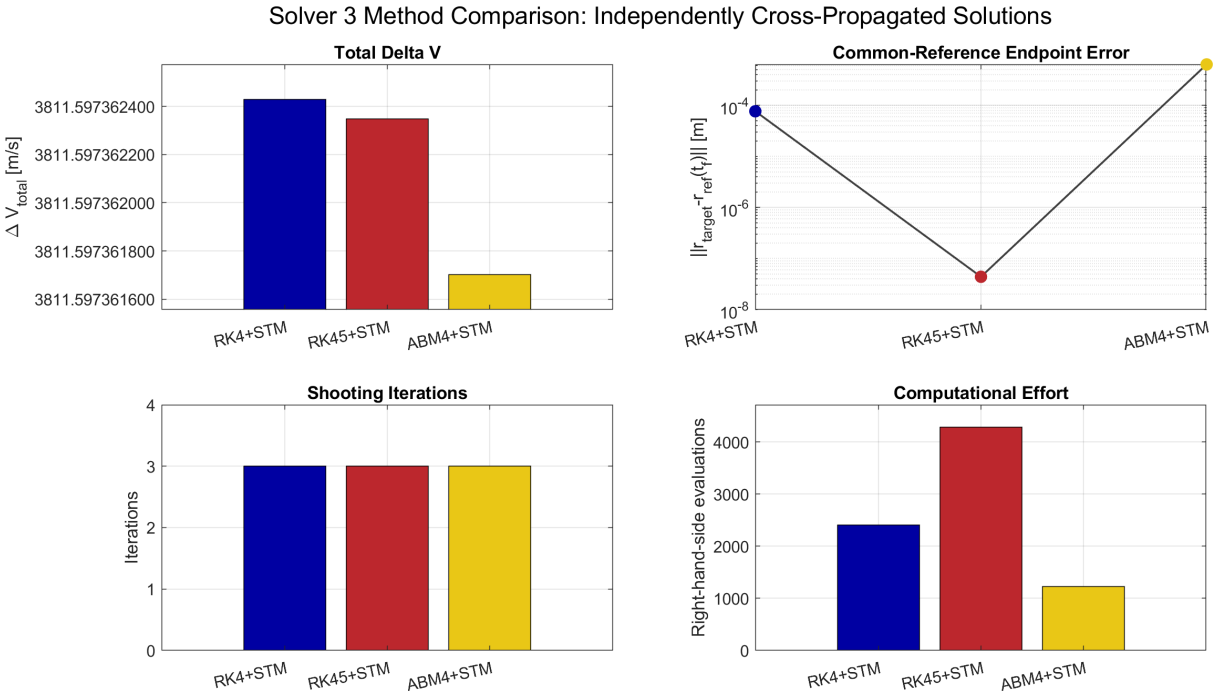


Figure 8: Comparison of Solver 3 numerical variants.

3.6 Overall Comparison of the Three Solvers

Figure 9 summarizes the total and component velocity increments obtained from the prescribed-time, optimized-time, and J_2-corrected transfers.

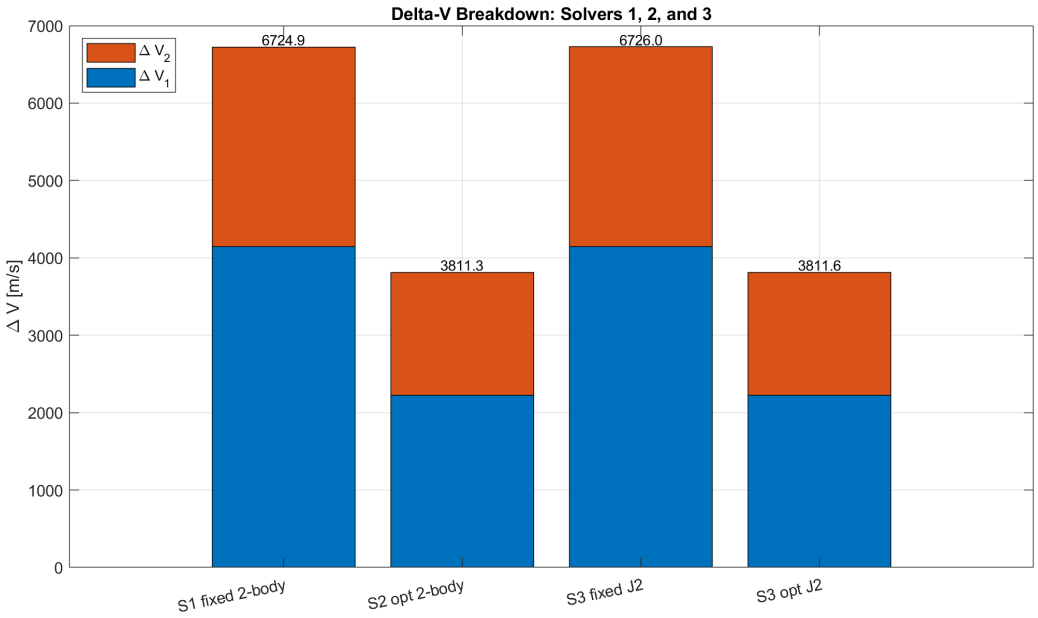


Figure 9: Total and component ΔV values for the two-body and J_2-corrected transfers.

The dominant design improvement resulted from transfer-time optimization. Moving from Solver 1 to Solver 2 reduced the total ΔV from 6724.946 m/s to 3811.291 m/s, a reduction of approximately 43.3%. By comparison, incorporating J_2 increased the fixed-time transfer cost by 1.022 m/s and the optimized-time transfer cost by only 0.306 m/s.

These results show that the two-body Lambert solution and transfer-time optimization determine the principal orbit geometry and maneuver cost, while Solver 3 provides the smaller but essential correction required for accurate endpoint targeting under perturbed propagation.

4 Numerical Analysis and Validation

Once the Lambert solver returns the initial and final velocity vectors $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$, the algorithm must be rigorously validated before it can be used for mission-planning decisions such as ΔV budgeting, propellant mass estimation, or launch window analysis.

Four validation checks were conducted — three physical consistency checks and one convergence diagnostic — that together constitute a complete numerical validation framework. All numerical benchmarks are derived from problem-scale dimensional analysis, machine-precision theory, and established results from numerical analysis literature (Burden and Faires, 2010; Higham, 2002).

4.1 Tolerances implemented in the three Solvers

The numerical tolerances and safety limits used directly in the MATLAB implementation are declared in Table 4. These values are internal controls that determine when an iteration stops or when a nearly singular expression is guarded. The validation benchmarks in the following sections are intentionally broader because they include accumulated effects from the Lambert iteration, orbital-element conversion, and endpoint differencing.

Table 4: Numerical tolerances and limits used in the MATLAB code

Code location	Quantity	Value	Purpose
`lambert_solver`	ToF residual	10^{-3} s	Stop when time error < 1 ms
`lambert_solver`	p step	10^{-10} m	Secondary stop criterion, preventing infinite loops
`lambert_solver`	Max p-iter	500	Safety cap against infinite loops
`lambert_solver`	$a(p)$ denom guard	$< 10^{-6}$	Avoid division by near-zero
`lambert_solver`	dt/dp guard	$< 10^{-12}$	Fallback when slope too small
`lambert_solver`	$k - lp$ guard	$< 10^{3}$	Prevent derivative instability
`lambert_solver`	Δp limiter	$\leq 0.5(p_{II} - p_I)$	Avoid large jumps
`lambert_solver`	p interval clamp	$1.0001p_I \leq p \leq 0.9999p_{II}$	Stay away from endpoints
`coe_from_sv`	Eccentricity floor	10^{-10}	Handle near-circular orbits as circular
`main_lambert`	Coincident pos. tol	10^{-9} m	Reject identical endpoints
`main_lambert`	Transfer angle tol	10^{-12} rad	Reject $\Delta\theta \approx 0, \pi$
`solver2`	Coarse grid samples	500	Scan for minimum basin
`solver2`	Local refinement	200	Refine best grid region
`solver2`	Golden-section tol	10^{-4} s	Stop interval below 0.1 ms
`solver2`	Brent `TolX`	10^{-6} s	High-resolution time
`solver2`	Brent `MaxFunEvals`	500	Limit Lambert calls
`solver2`	Search interval margin	$1.01dt_{\min}$, $0.99dt_{\max}$	Avoid singular bounds
`SOLVER3`	RK4 step size S	`10 s`	Fixed-step augmented (42-state) propagation

Table 4: Continued

Code location	Quantity	Value	Purpose
SOLVER3	Augmented state dim.	42	6 state + 36 STM entries propagated jointly
SOLVER3	ode45 tolerances	RelTol = 10^{-12}; component-wise AbsTol	Adaptive RK45 validation
SOLVER3	ABM4 nominal step	10 s	Uniform predictor–corrector validation steps
SOLVER3	Shooting residual	10^{-7} m	Stop the discrete STM correction
SOLVER3	Reference propagation	RelTol = 2.5×10^{-13}, $S_{\max} = 0.25$ s	Cross-check the converged departure velocity
SOLVER3	Max shooting iter	20	Safety cap on Newton corrections
SOLVER3	Final RK4 step trim	$\min(S, t_f - t)$	Land exactly on t_f

4.2 Numerical Validation of lambert solver

The validation benchmarks in Table 5 judge the physical and algorithmic quality of the final solution. A validation tolerance is not identical to a code tolerance; for example, the Lambert solver stops at a time-of-flight residual of 10^{-3}s, but the position, energy, and angular-momentum checks include additional calculations after the iteration ends. Their acceptable thresholds allow for this post-processing while remaining strict enough to detect a failed transfer.

Table 5: Validation benchmarks with derivation and engineering effect

	Primary diagnostic	Physical concept	Acceptable benchmark	Benchmark effect	Derivation source
1	Position closure error $\|\hat{\mathbf{r}}_2 - \mathbf{r}_2\|$	Propagated trajectory reaches $\mathbf{r}_2$	< 10m	Accommodates propagation and iteration residuals; 10m is negligible for mission planning. (Carpenter and Alfriend, 2003)	Theoretical floor $\sim 6 \times 10^{-8}$m; relaxed to 10m for Kepler propagation and anomaly errors.

Table 5: Continued

	Primary diagnostic	Physical concept	Acceptable bench-mark	Benchmark effect	Derivation source
2	Transfer arc endpoints specific-energy difference $\lvert\varepsilon_1 - \varepsilon_2\rvert$	Both endpoints lie on the same Keplerian ellipse	< 15J/kg	Energy conservation to $\sim 10^{-6}$ relative is adopted as a conservative validation threshold; equivalent to velocity error 0.003m/s.	Derived from 10^{-6} relative error applied to characteristic energy 1.5×10^7J/kg (approximately half the specific energy of a typical LEO, as a conservative lower bound).
3	Relative angular-momentum difference $\frac{\lvert\mathbf{h}_1 - \mathbf{h}_2\rvert}{\lvert\mathbf{h}_1\rvert}$	Both states in the same orbital plane	$< 10^{-6}$	Allows small numerical noise while rejecting physically inconsistent plane rotations.	Theoretical relative floor $\sim 6.6 \times 10^{-16}$; relaxed to 10^{-6} for near-singular geometries.
4	p-iteration: iterations and order α	Newton conver-gence for $t(p) = dt$	Iterations ≤ 15 and $\alpha \geq 1.5$	Provides conservative safety margin and ensuring a minimum convergence order above the ideal quadratic convergence.	$e_0 \sim 10^3$ s and tolerance 10^{-3} s give $N_{\text{req}} \approx 2.6$; 15 iterations provide a conservative margin. Newton–Raphson is ideally quadratic ($\alpha \approx 2$), though numerical effects may reduce α.

4.2.1 Validation Test Cases of solver1

The validation checks were applied to the fixed-time Lambert solver (Solver1) using a set of representative test cases. The results are summarised in Table 6. Each case examines a different geometric or temporal regime, including nominal transfers, near-singular geometries, and long transfer times.

Table 6: Solver1 validation results for five test cases. red=FAIL, orange=CHECK.

Case	r_1 (km)	r_2 (km)	dt (s)	Check1 (m)	Check2 (J/kg)	Check3 (m^2/s^2)	Check4 (iter / α / ToF res)
Normal passing	$[5000, 10000, 0]$	$[14000, 5000, 0]$	3600	2.61×10^{-5}	3.73×10^{-9}	1.53×10^{-5}	4 / 1.990 / 8.34×10^{-9}
Very short time	$[5000, 10000, 0]$	$[14000, 5000, 0]$	60	lambert_solver failed after 500 iterations			
Near-180° transfer	$[7000, 0, 0]$	$[-6999, 1, 0]$	3600	lambert_solver failed after 500 iterations			
Near-zero-angle transfer	$[7000, 0, 0]$	$[7100, 10, 0]$	10	3.03×10^{-1}	2.61×10^{-7}	9.54×10^{-7}	10 / 1.383 / 3.03×10^{-5}
Long branch	$[5000, 10000, 0]$	$[14000, 5000, 0]$	12000	8.30×10^{6}	9.31×10^{-9}	1.14×10^{-5}	7 / 1.683 / 5.06×10^{-10}

The **normal transfer case shows excellent solver performance**, with all validation

checks far below the acceptable thresholds and near-quadratic convergence ($\alpha \approx 1.99$). This confirms that Solver1 performs reliably for well-conditioned Lambert transfers.

For the very short transfer-time case and the near-180° transfer case, the solver fails to converge within the maximum allowed number of iterations. The short transfer-time case is particularly challenging for classical p-iteration methods because the solution approaches the minimum-time boundary, where the Lambert equations become highly nonlinear and poorly conditioned. In the near-180° case, the transfer geometry approaches $\Delta\theta \approx \pi$, a region known to cause numerical difficulties and slow convergence. **These failures are expected** for the implemented elliptic, short-way, single-revolution p-iteration formulation, and demonstrate that the solver correctly rejects unsupported problems.

The near-zero-angle transfer converges successfully, but with slower convergence ($\alpha = 1.38$), indicating that the p-iteration becomes nearly linear for very small transfer angles. Nevertheless, all physical consistency checks remain within acceptable engineering tolerances.

For the long transfer-time branch, the solver satisfies the algebraic checks (energy and angular momentum conservation) but fails the position-closure check with a very large error. This indicates that the returned solution belongs to a different Lambert branch than the intended transfer.

4.2.2 Validation Test Cases of solver2

Solver2 was evaluated using validation checks as used in solver1 to examine its behavior under both nominal and challenging conditions. The test suite spans conventional orbital transfers, near-degenerate geometries, and large-radius transfers in order to verify solution accuracy, convergence properties, and algorithm robustness. The corresponding results are presented in Table 7.

Table 7: Solver2 validation results for five test cases. red=FAIL, orange=CHECK.

Case	r_1 (km)	r_2 (km)	dt optimal(s)	Check1 (m)	Check2 (J/kg)	Check3 (m^2/s^2)	Check4 (iter / α / ToF res)
Normal passing	$[5000, 10000, 0]$	$[14000, 5000, 0]$	2002.909999	1.36×10^{-1}	3.73×10^{-9}	0.00	6 / 1.756 / 2.98×10^{-5}
Circular quarter- orbit case	$[7000, 0, 0]$	$[0, 7000, 0]$	1457.129	7.245	0.00	0.00	5 / 1.802 / 9.601×10^{-4}
Near-180° transfer	$[7000, 0, 0]$	$[-6999, 1, 0]$	no valid Lambert solutions were found in the search interval				
Near-zero-angle transfer	$[7000, 0, 0]$	$[7100, 10, 0]$	297.499	5.11×10^{-3}	0.00	0.00	11 / 1.378 / 6.138×10^{-6}
Nearly radial LEO to GEO	$[6578, 0, 0]$	$[42164, 100, 0]$	14801.173	2.932×10^{-8}	1.49×10^{-8}	0.00	3 / 1.999 / 1.139×10^{-8}

The normal transfer case demonstrates good solver performance, with all validation checks remaining well within the prescribed tolerances, indicating stable superlinear convergence for a well-conditioned Lambert transfer.

The circular quarter-orbit case also converges successfully and provides physically consistent results. Although the position-closure error increases to approximately 7.2 m, the energy and angular-momentum checks remain effectively zero.

For the near-180° transfer case, no valid Lambert solutions were found within the search interval. Transfer geometries approaching $\Delta\theta \approx \pi$ are known to be numerically challenging because multiple solution branches merge and the time-of-flight function becomes highly sensitive to small changes in the transfer parameter. The absence of a valid solution in the search interval is therefore **expected** and reflects the limitations of the solution.

The near-zero-angle transfer converges successfully but exhibits slower convergence, with an observed convergence order of $\alpha = 1.378$. This behavior is consistent with the reduced numerical sensitivity of the time-of-flight function for very small transfer angles.

The nearly radial LEO-to-GEO transfer case represents a demanding transfer geometry due to the large difference in orbital radius and the nearly collinear position vectors. Despite these challenging conditions, the solver converges in only three iterations with an observed convergence order of $\alpha = 1.999$, indicating essentially quadratic convergence. All validation metrics remain near machine precision. This result demonstrates the excellent numerical stability and accuracy of Solver 2 for long-duration, nearly radial transfer geometries.

4.3 Numerical-Error Validation of Solver 3

Whereas Solver 1 and Solver 2 RE validated through the physical-consistency and convergence checks of Section 4.2, Solver 3 introduces three additional numerical components that require independent verification: the analytical gravity-gradient Jacobian $G = \partial a/\partial r$, the State Transition Matrix (STM) propagated alongside the state, and the Newton shooting iteration. Each was tested against quantitative pass criteria drawn from the numerical-analysis and orbital mechanics literature. All tests use the equatorial mission scenario of Table 3; an additional inclined case (the same radial magnitudes tilted 25° out of plane) was run in parallel to exercise the $a_z(z \neq 0)$ code path that the equatorial geometry would otherwise mask.

Analytical Jacobian. The gravity gradient G must be symmetric for a conservative field. Comparison against a centred finite-difference Jacobian at four representative points (equatorial r_1, equatorial r_2, inclined LEO, and polar) gave a symmetry residual $\max |G - G^{\mathsf{T}}| < 10^{-22}$ and an error versus finite difference below 3×10^{-16}, i.e. at machine epsilon. The trace of G at the polar point vanishes to machine precision, consistent with $\nabla \cdot a = 0$ away from the central mass. The analytical component expressions are used directly in the implementation; their symmetry is verified numerically rather than enforced through an additional matrix-symmetrisation operation.

State Transition Matrix. Three independent tests were applied at the converged shooting solution, the point at which the Newton step actually uses the Φ_{rv} block. A finite-difference cross-check, sweeping the relative perturbation over $[10^{-2}, 10^{-12}]$, produced the characteristic V-shaped error curve and reached a best relative agreement of 6.3×10^{-11} near the optimal perturbation $\epsilon \approx 10^{-5}$, where the $O(\epsilon^2)$ truncation and $\epsilon_{\text{mach}}/\epsilon$ round-off branches balance. The symplecticity residual was $\|\Phi^{\mathsf{T}} J \Phi - J\|_F = 1.4 \times 10^{-10}$ with $|\det \Phi - 1| = 3.7 \times 10^{-14}$, showing that phase-space volume is preserved to fourteen digits despite RK4 not being a symplectic integrator. The group property $\Phi(t_2, t_0) = \Phi(t_2, t_1)\,\Phi(t_1, t_0)$, evaluated at the midpoint $t_1 = \Delta t/2$, held to a relative error of 3.2×10^{-16}. Results are summarised in Table 8.

Integrator convergence. The three corrected departure velocities were cross-propagated using the common reference defined in Equation 80. For the fixed-step methods, the nominal step was successively halved from 20 to 10 and 5 s. The RK4 endpoint error decreased by factors of 16.1 and 16.2, matching the theoretical fourth-order factor $2^4 = 16$. The ABM4 error decreased by factors of 13.7 and 14.9, approaching the same asymptotic fourth-order rate. RK45 does not follow a fixed-step power law because its internal step

Table 8: Solver3 STM validation tests.

Test	Quantity	Measured	Threshold
A	best $\|\Phi_{\text{an}} - \Phi_{\text{FD}}\|_F / \|\Phi_{\text{an}}\|_F$	6.3×10^{-11}	$\leq 10^{-7}$
B	$\|\Phi^{\mathsf{T}} J \Phi - J\|_F$	1.4×10^{-10}	$\leq 10^{-7}$
C	$\|\det \Phi - 1\|$	3.7×10^{-14}	$\leq 10^{-10}$
D	group-property relative error	3.2×10^{-16}	$\leq 10^{-10}$

size is selected from the error tolerances and $S_{\max}$ is only an upper bound, not the actual integration step. It is therefore excluded from the fixed-step convergence plot and assessed separately in the method comparison. At the nominal 10 s setting, the conservative cross-propagated errors were 7.66×10^{-5} m for RK4, 4.39×10^{-8} m for RK45, and 6.33×10^{-4} m for ABM4.

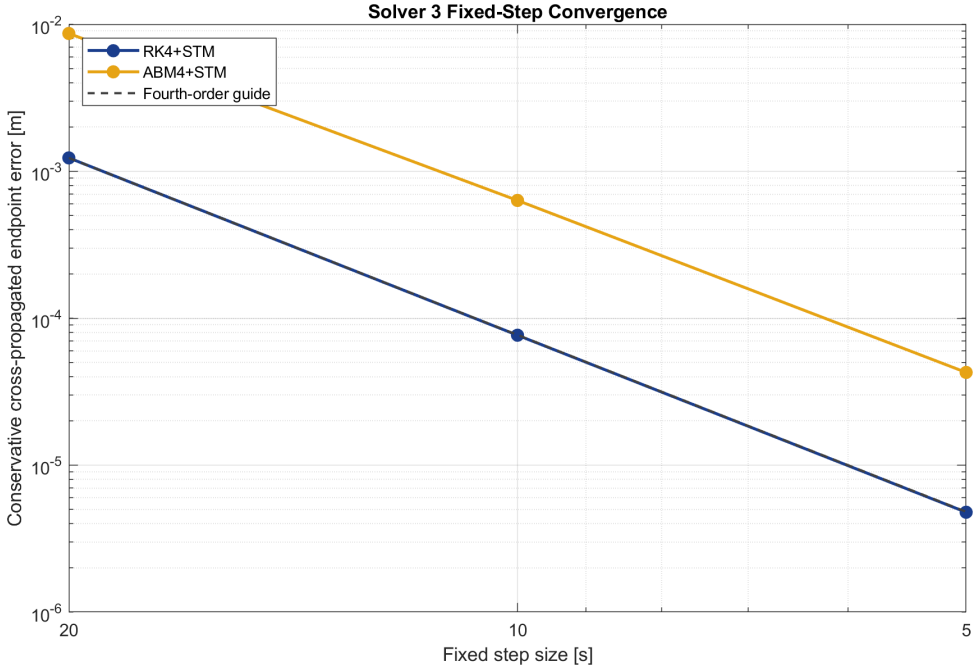


Figure 10: Step-size convergence of the fixed-step propagation variants.

Shooting iteration. For the prescribed-time case, the Newton–STM procedure converged in three residual evaluations, corresponding to two departure-velocity corrections. The internal residual decreased from 6.774×10^{3} m to 7.038×10^{-1} m and then to 4.165×10^{-9} m, as shown in Figure 11. The optimized-time case also required three residual evaluations, decreasing from 2.466×10^{3} m to 5.398×10^{-2} m and then to 5.665×10^{-9} m. Table 9 separates these internal convergence residuals from the independently cross-propagated endpoint errors.

Taken together, these tests establish that every numerical component of Solver 3 behaves as theory predicts. For the mission scenario, including J_2 raises the total velocity increment by 1.02 m/s in the prescribed-time case. The optimized-time RK4 solution has a conservative cross-propagated endpoint error of 7.66×10^{-5} m, while the adaptive RK45 validation reaches the common-reference resolution floor below 10^{-7} m. The distinction between the

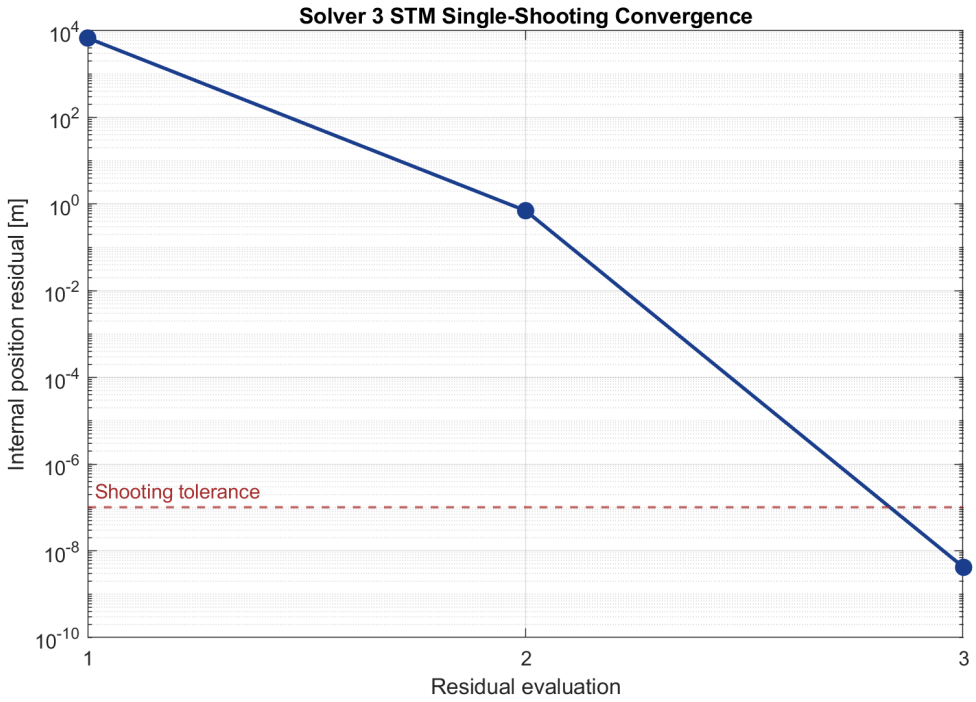


Figure 11: Solver 3 shooting-method convergence.

Table 9: Solver 3 comparison of augmented-state propagation methods for the optimized transfer case.

Method	Iter.	RHS eval.	Shooting residual (m)	Reference error (m)
RK4 + STM	3	2412	5.67×10^{-9}	7.66×10^{-5}
RK45 + STM	3	4287	1.34×10^{-8}	4.39×10^{-8}
ABM4 + STM	3	1227	1.67×10^{-8}	6.33×10^{-4}

internal shooting residual and the cross-propagated error prevents the discrete Newton convergence from being mistaken for propagation accuracy.

4.4 Comparative analysis with established Lambert algorithms

To validate the developed TPP Lambert solver, a comprehensive comparison was performed against three established Lambert algorithms from the orbital dynamics literature: Gooding's method, Battin's method, and Sun's method. The objective of this analysis is to evaluate the numerical accuracy, sensitivity to transfer parameters, and robustness of the TPP solver relative to well-established reference methods. All tests use the same parameters of the scenario in Table 3.

The validation campaign consisted of three independent tests:

1. Time-of-flight (TOF) sensitivity analysis.
2. Transfer-angle sensitivity analysis.
3. Near-180° geometry robustness analysis.

4.4.1 Reference Lambert Algorithms

Three reference Lambert algorithms were selected for comparison (Eagle, 2024). They differ in their numerical formulation, iteration strategy, and range of applicability, making them suitable for assessing both accuracy and robustness of the proposed implementation across a wide spectrum of orbital transfer scenarios.

Table 10: Comparison of Lambert algorithms

Solver	Main Variable	Iteration Method	Orbit Types	Strengths	Weaknesses
`lambert_gooding` (Gooding, 1990)	Dimensionless Parameter x (eccentricity related)	Halley iteration	Elliptic, Parabolic, Hyperbolic	Extremely robust, benchmark-quality, very accurate near-parabolic treatment	Complex initial guess splicing
`lambert_battin` (Battin and Vaughan, 1984)	Parameter x (continued fractions)	Successive substitution	Elliptic, Hyperbolic	Excellent numerical stability, robust for all conic sections; uses pre-computed coefficients for fast convergence	Does not handle parabolic orbits explicitly, simpler initial guess may fail in some cases
`lambert_sun` (Sun, 1979)	Dimensionless Parameter x (eccentricity related)	Laguerre iteration	Elliptic, Parabolic, Hyperbolic	Fast convergence; handles all conic types, uses analytic derivatives	Laguerre may be less robust than Battin or Gooding for some regimes
`lambert_tpp`	Semi-latus rectum p	Newton P-iteration	Elliptic only	Very simple, intuitive formulation; direct computation of Lagrange coefficients; educational value	Only short-way transfer ($\Delta\theta \leq \pi$); elliptic only, no multi-rev support

Agreement between the TPP algorithm and these established algorithms provides strong evidence of the correctness and reliability of the implementation.

4.4.2 Time-of-Flight Sensitivity Analysis

The first validation test examined the consistency of the computed Lambert solutions as the transfer time was varied while maintaining a fixed transfer geometry.

The discrepancy between the TPP solver and each reference solver was quantified using the combined departure and arrival velocity error of the transfer orbit.

$$\varepsilon_V = \sqrt{\|\mathbf{v}_{1,\mathrm{TPP}} - \mathbf{v}_{1,\mathrm{ref}}\|^2 + \|\mathbf{v}_{2,\mathrm{TPP}} - \mathbf{v}_{2,\mathrm{ref}}\|^2} \tag{81}$$

The complete numerical results are provided in Table 13 of Appendix A.

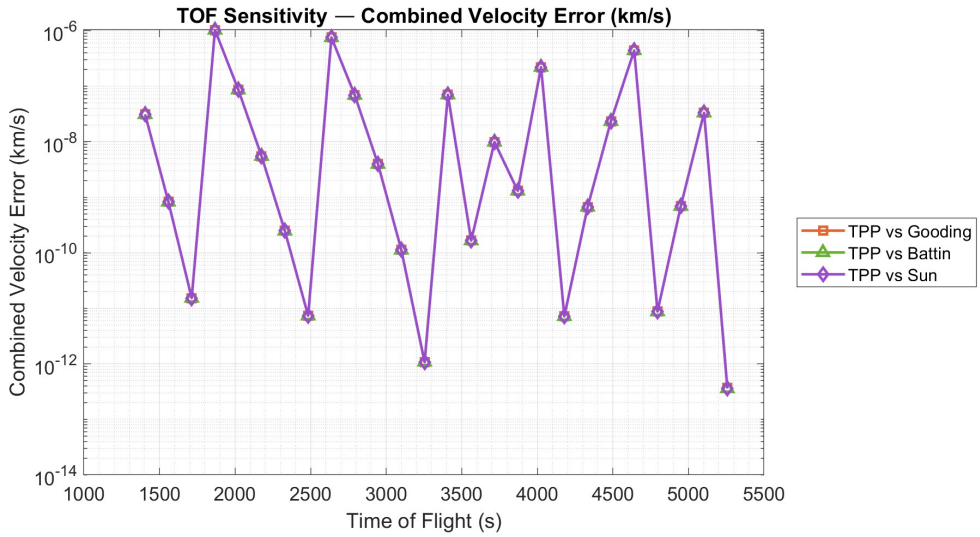


Figure 12: Combined velocity error versus time of flight.

The three comparison curves are nearly indistinguishable over the entire tested TOF range. The errors remain extremely small and do not exhibit any systematic increase with increasing transfer duration.

Small oscillations are visible in the curves; however, these variations are several orders of magnitude below practical mission-design tolerances and are primarily attributable to numerical round-off effects. The maximum discrepancy observed throughout the TOF sweep remained on the order of 10^{-6} km/s or smaller.

These results demonstrate that the TPP solver consistently converges to the same physical solution obtained by Gooding, Battin, and Sun for a wide range of transfer times.

4.4.3 Transfer-Angle Sensitivity Analysis

To investigate solver agreement across different orbital geometries, the transfer angle was varied from 10° to 170° while maintaining a fixed time of flight of 3600 s.

The complete numerical results are provided in Table 14 of Appendix A.

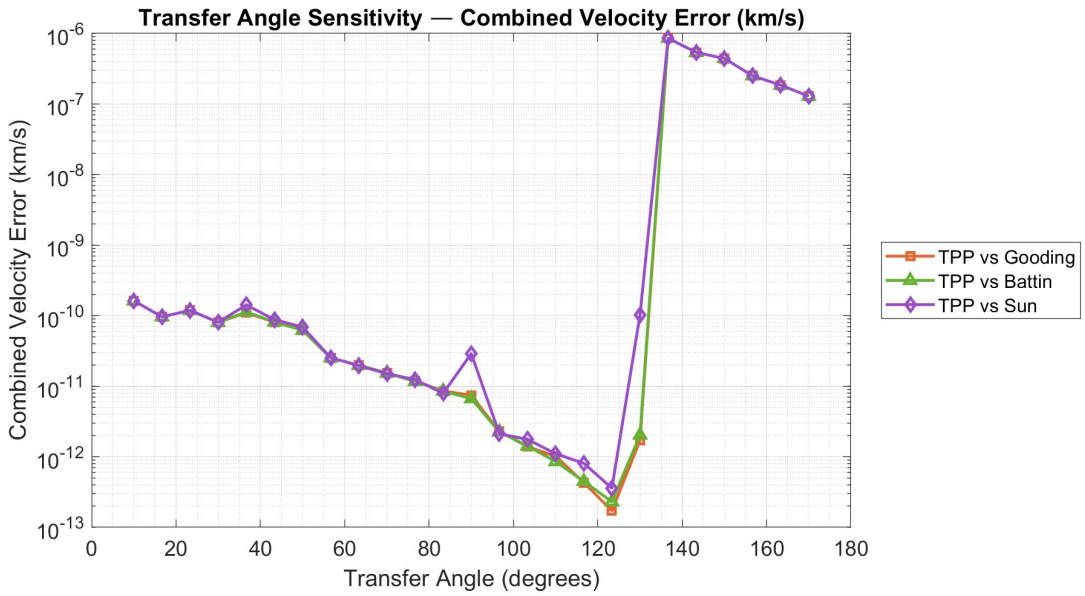


Figure 13: Combined velocity error versus transfer angle.

Three distinct regions can be identified in the figure. For transfer angles below approximately 130°, the discrepancy remains near machine precision and is effectively negligible. Near 135°, a localized increase in the error is observed, producing the largest discrepancy of the entire transfer-angle sweep.

Beyond this region, the discrepancy gradually decreases as the transfer angle approaches 170°. Although the increase around 135° is clearly visible on the logarithmic scale, the absolute magnitude remains extremely small, approximately 10^{-6} km/s.

Consequently, the transfer-angle sensitivity analysis confirms that the TPP solver maintains excellent agreement with the reference methods over a broad range of orbital geometries.

4.4.4 Near-180° Transfer

The most challenging numerical configuration for Lambert solvers occurs when the transfer angle approaches 180°. In this region, many formulations suffer from numerical conditioning problems that can lead to inaccurate solutions or convergence failures.

To assess solver robustness, a dedicated study was performed for transfer angles ranging from 160° to 179°.

The complete numerical results are provided in Table 15 of Appendix A.

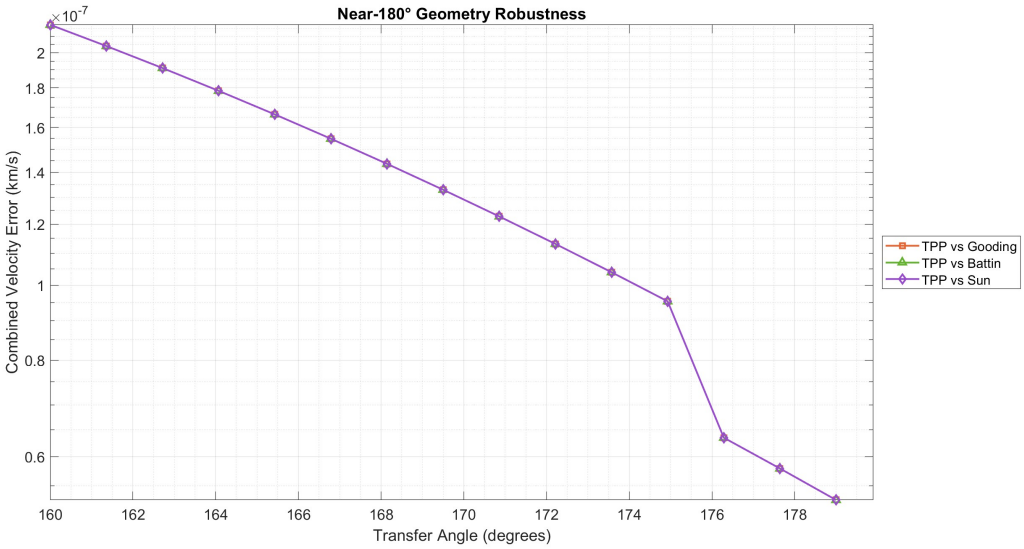


Figure 14: Solver agreement for transfer angles approaching the critical 180° geometry.

The results demonstrate that the discrepancy between the TPP solver and the reference algorithms remains **extremely small** throughout the tested range. Furthermore, the error gradually decreases as the transfer angle approaches 180°.

No convergence failures were observed during this analysis, and all solutions remained in **excellent agreement** with Gooding, Battin, and Sun. The maximum discrepancy remained on the order of 10^{-6} km/s and decreased to nearly machine precision at the largest transfer angles.

5 Visualization

5.1 Cross-Verification of Transfer Orbits

To verify the practical implementation of the Lambert transfer solutions, the trajectories computed by Solver1, Solver2 and Solver3 were simulated in two independent orbital mechanics software packages: GMAT (General Mission Analysis Tool) and STK (Systems Tool Kit). Both tools were configured with identical initial conditions and impulsive ΔV manoeuvres. The resulting transfer orbits were compared against the theoretical two-body elements computed by the MATLAB TPP solvers.

5.2 Simulation via GMAT

Figure 15 shows the transfer trajectories (red orbit) implemented by GMAT for the three solvers.

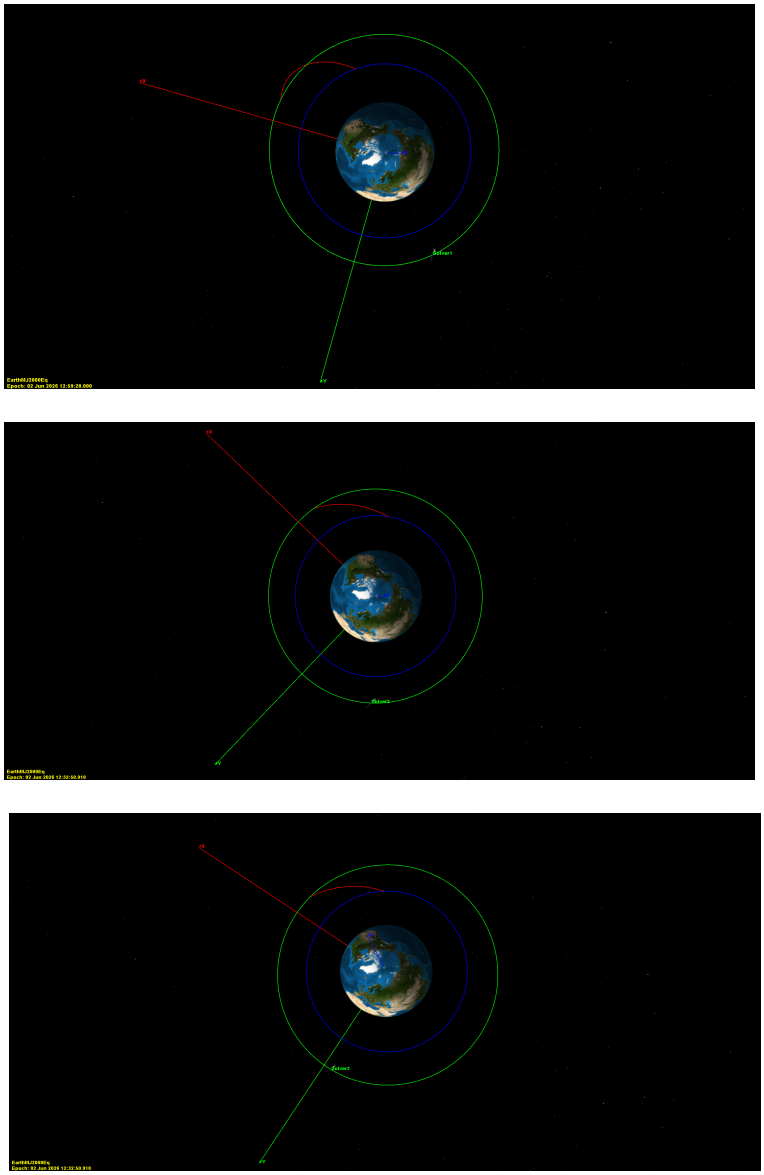


Figure 15: GMAT visualisation of the transfer orbits: Solver 1 (top), Solver 2 (middle), Solver 3 (bottom).

The orbital elements extracted from the propagated states are summarised in Table 11.

Table 11: Orbital elements of the transfer orbits computed in GMAT

Parameter	**Solver1**	**Solver2**	**Solver3**
Transfer time dt [s]	3600.000	2002.90999	2002.90999
Angular momentum [km^2/s]	42971.953	64844.163	64846.760
Eccentricity e	0.700	0.365	0.365
Inclination i [deg]	180	180	180
Semimajor axis a [km]	9088.486	12172.032	12174.632
Orbital period [s]	8622.799	13364.590	13368.872

5.3 Simulation via STK

The same transfer orbits were then implemented in STK. The resulting 3-D visualisations are presented in Figure 16, and the extracted orbital elements are listed in Table 12.

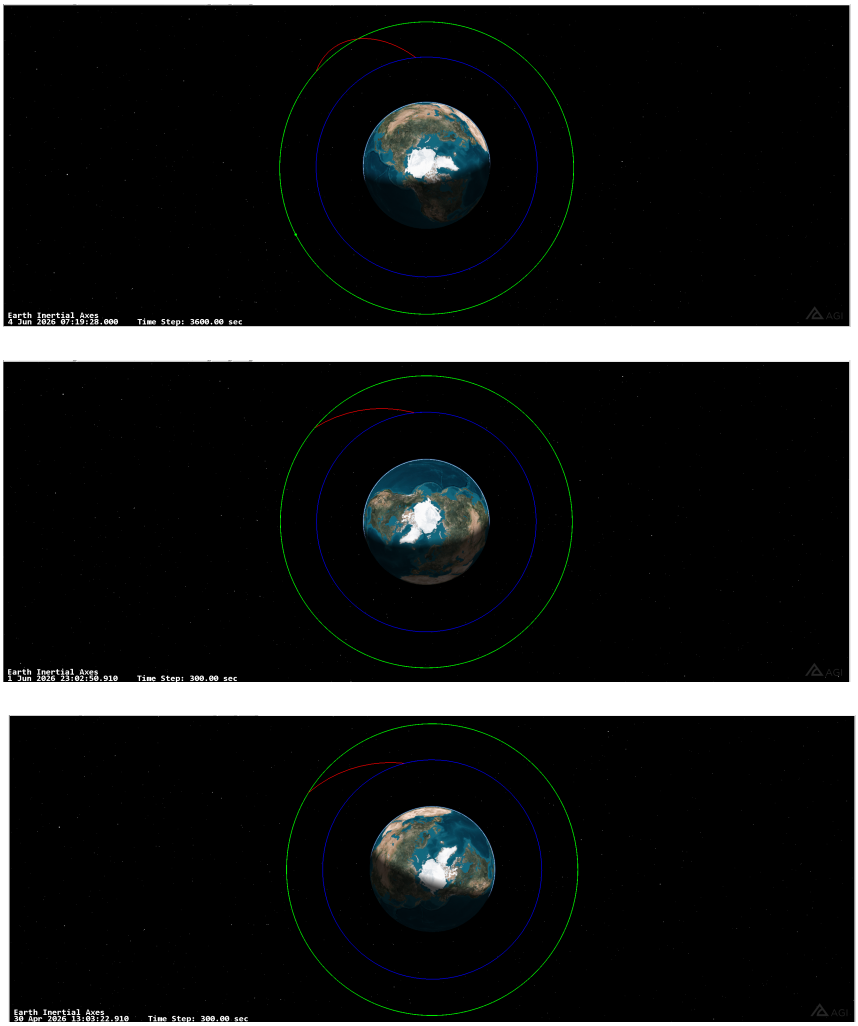


Figure 16: STK visualisation of the transfer orbits: Solver1 (top), Solver2 (middle), Solver3 (bottom).

Table 12: Orbital elements of the transfer orbits computed in STK

Parameter	**Solver 1**	**Solver 2**	**Solver 3**
Transfer time dt [s]	3600.000	2002.90999	2002.90999
Angular momentum [km^2/s]	42972.020	64844.215	64848.505
Eccentricity e	0.700	0.365	0.365
Inclination i [deg]	180	180	180
Semimajor axis a [km]	9088.490	12172.033	12171.947
Orbital period [s]	8622.806	13364.591	13364.448

5.4 Discussion of Cross-Verification

The orbital elements obtained from GMAT and STK agree very closely with each other and with the theoretical MATLAB values. Table 11 and Table 12 show that for Solver1, Solver2, and Solver3, all parameters match within very tight tolerances. These minor discrepancies can be attributed to:

- **Numerical integration:** GMAT uses an adaptive Runge-Kutta 4/5 method (tolerance 10^{-12}), while STK uses a fixed-step method (60 s). This leads to slightly different accumulated truncation errors (Higham, 2002).
- **Round-off and coordinate system conversions:** Converting ΔV to VNB/VNC axes involves floating-point operations that introduce errors in position (Burden and Faires, 2010).

The overall close agreement in most orbital elements successfully validates the end-to-end implementation of the transfer algorithms and their integration into mission-level software (GMAT and STK).

6 Conclusion

This project successfully developed and validated a reusable MATLAB framework for solving short-way, single-revolution elliptic Lambert transfer problems. The principal achievements of the three solvers are summarized below.

Solver Achievements

- **Solver 1 – Prescribed-Time Lambert Transfer:** The two-body p-iteration solver determined the transfer velocities and orbital elements for the prescribed transfer time of 3600 s. It obtained a semi-major axis of 9088.497 km and a total velocity increment of 6724.946 m/s.
- **Solver 2 – Transfer-Time Optimization:** Grid refinement, golden-section search, and Brent's method identified an optimal transfer time of approximately 2002.910 s. The resulting total velocity increment was 3811.291 m/s, representing a reduction of approximately 43.3% relative to the prescribed-time solution.
- **Solver 3 – J_2 Perturbation Correction:** RK4 propagation, State Transition Matrix propagation, and Newton shooting corrected the two-body solution under the J_2 perturbation model. The method converged in three iterations and reduced kilometre-scale initial targeting errors to numerical shooting residuals below 6×10^{-9} m.

Lessons Learned

- **Transfer time strongly influences maneuver cost.** Optimizing only the transfer duration produced a much larger improvement in total ΔV than the subsequent perturbation correction.
- **Perturbations are important for accurate endpoint targeting.** The J_2 effect changed the total ΔV by only a small amount, but neglecting it caused kilometre-scale terminal-position errors during propagation.
- **Independent validation increases confidence in numerical results.** Comparisons between optimization methods, established Lambert algorithms, GMAT, STK, and numerical consistency checks helped distinguish reliable solutions from incorrect Lambert branches.
- **A modular program structure improves development and verification.** Separating the core Lambert algorithm, optimization methods, orbital-element calculations, and J_2 correction simplified testing, comparison, and future extension of the MATLAB framework.
- **Teamwork was essential for integrating the project components.** Dividing the development, perturbation modelling, optimization, validation, and visualization tasks among team members enabled the three solvers to be combined into one consistent framework.

Overall, the developed framework achieved the required classical Lambert solution, transfer-time optimization, and J_2-perturbed correction. Future developments should include Earth-clearance constraints, long-way and multi-revolution transfers, additional perturbation models, and improved handling of near-degenerate geometries.

References

Arora, N. and Russell, R. P. (2013). A fast and robust multiple-revolution Lambert algorithm using a cosine transformation. *AAS/AIAA Paper AAS 13-728*.

Avanzini, G. (2008). A simple Lambert algorithm. *Journal of Guidance, Control, and Dynamics*, 31(6):1587–1594.

Bate, R. R., Mueller, D. D., and White, J. E. (1971). *Fundamentals of Astrodynamics*. Dover Publications, New York.

Battin, R. H. (1999). *An Introduction to the Mathematics and Methods of Astrodynamics*. AIAA Education Series. AIAA, Reston, VA, revised edition.

Battin, R. H. and Vaughan, R. M. (1984). An elegant Lambert algorithm. *Journal of Guidance, Control, and Dynamics*, 7(6):662–670.

Brent, R. P. (1973). *Algorithms for Minimization Without Derivatives*. Prentice-Hall, Englewood Cliffs, NJ. Chapter 5 describes the hybrid golden-section/parabolic-interpolation method implemented in MATLAB's `fminbnd`.

Burden, R. L. and Faires, J. D. (2010). *Numerical Analysis*. Brooks/Cole, Boston, MA, 9 edition.

Carpenter, J. R. and Alfriend, K. T. (2003). Navigation accuracy guidelines for orbital formation flying missions. Technical Report NASA/TP-2003-212236, NASA Goddard Space Flight Center.

Conte, D. (2014). Determination of optimal Earth-Mars trajectories to target the moons of Mars.

Curtis, H. D. (2019). *Orbital Mechanics for Engineering Students*. Aerospace Engineering. Butterworth-Heinemann, Oxford, 4 edition.

De la Torre Sangrà, D. and Fantino, E. (2015). Review of Lambert's problem. In *ISSFD 2015: 25th International Symposium on Space Flight Dynamics*, Munich.

Dormand, J. R. and Prince, P. J. (1980). A family of embedded runge-kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26.

Eagle, D. (2024). Matlab functions for solving lambert's problem. MATLAB Central File Exchange. Accessed: 2026-06-03, Available at: https://www.mathworks.com/matlabcentral/fileexchange/158221-matlab-functions-for-solving-lambert-s-problem.

Gauss, C. F. (1809). *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. F. Perthes and I. H. Besser, Hamburg.

Gooding, R. H. (1990). A procedure for the solution of Lambert's orbital boundary-value problem. *Celestial Mechanics and Dynamical Astronomy*, 48(2):145–165.

He, S. (2010). Analytical derivatives for eccentricity-vector-based Lambert problem. *Journal of Guidance, Control, and Dynamics*.

Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2 edition.

Hohmann, W. (1925). *Die Erreichbarkeit der Himmelskörper*. Oldenbourg, Munich. English translation: *The Attainability of Heavenly Bodies*, NASA Technical Translation F-44, 1960.

Hu, W. (2015). *Fundamental Spacecraft Dynamics and Control*. John Wiley & Sons, Singapore.

Izzo, D. (2015). Revisiting Lambert's problem. *Celestial Mechanics and Dynamical Astronomy*, 121(1):1–15.

Kiefer, J. (1953). Sequential minimax search for a maximum. *Proceedings of the American Mathematical Society*, 4(3):502–506.

Kluever, C. A. (2018). *Space Flight Dynamics*. John Wiley & Sons, Hoboken, NJ.

Kutta, W. (1901). Beitrag zur näherungsweisen integration totaler differentialgleichungen. *Zeitschrift für Mathematik und Physik*, 46:435–453.

Lagrange, J.-L. (1780). Sur une manière particulière d'exprimer le temps dans les sections coniques, décrites par des forces tendantes au foyer et réciproquement proportionnelles aux carrés des distances. *Nouveaux mémoires de l'Académie royale des sciences et belles-lettres*, 1778:181–202.

Lambert, J. H. (1761). Personal letter to Leonhard Euler. Archival correspondence referenced in Gauss, *Theoria Motus*, 1809, and in modern Lambert problem literature.

Lancaster, E. R. and Blanchard, R. C. (1969). A unified form of Lambert's theorem. Technical Report TN D-5368, NASA.

McElreath, J., Down, I. M., and Majji, M. (2025). A universal approach for solving the multi-revolution Lambert's problem. *Celestial Mechanics and Dynamical Astronomy*, 137(22).

Nocedal, J. and Wright, S. J. (2006). *Numerical Optimization*. Springer, New York, 2 edition.

Runge, C. (1895). Über die numerische auflösung von differentialgleichungen. *Mathematische Annalen*, 46(2):167–178.

Russell, R. P. (2019). On the solution to every Lambert problem. *Celestial Mechanics and Dynamical Astronomy*, 131(50).

Shampine, L. F. and Reichelt, M. W. (1997). The matlab ode suite. *SIAM Journal on Scientific Computing*, 18(1):1–22.

Simó, C. (1973). Solving Lambert's problem by regularization. Unpublished manuscript, University of Barcelona.

Sun, F. T. (1979). On the minimum-time trajectory and multiple solutions of Lambert's problem. In *AAS/AIAA Astrodynamics Specialist Conference*, Provincetown, USA.

Vallado, D. A. (2013). *Fundamentals of Astrodynamics and Applications*. Space Technology Library. Microcosm Press, Hawthorne, CA, 4 edition.

A Raw Numerical Validation Tables

This appendix collects the raw numerical tables from the Lambert-solver comparison studies. The main text keeps the corresponding trend figures and interpretation, while the full numerical tables are retained here for traceability.

Table 13: Raw numerical results of the TOF sensitivity analysis.

TOF(s)	Diff_TPP_vs_God	Diff_TPP_vs_Bat	Diff_TPP_vs_Sun	Mean_Diff
1405.0	0.000000	0.000000	0.000000	0.000000
1559.1	0.000000	0.000000	0.000000	0.000000
1713.1	0.000000	0.000000	0.000000	0.000000
1867.2	0.000013	0.000013	0.000013	0.000013
2021.3	0.000001	0.000001	0.000001	0.000001
2175.4	0.000000	0.000000	0.000000	0.000000
2329.5	0.000000	0.000000	0.000000	0.000000
2483.6	0.000000	0.000000	0.000000	0.000000
2637.7	0.000012	0.000012	0.000012	0.000012
2791.8	0.000001	0.000001	0.000001	0.000001
2945.9	0.000000	0.000000	0.000000	0.000000
3100.0	0.000000	0.000000	0.000000	0.000000
3254.1	0.000000	0.000000	0.000000	0.000000
3408.2	0.000001	0.000001	0.000001	0.000001
3562.3	0.000000	0.000000	0.000000	0.000000
3716.4	0.000000	0.000000	0.000000	0.000000
3870.4	0.000000	0.000000	0.000000	0.000000
4024.5	0.000004	0.000004	0.000004	0.000004
4178.6	0.000000	0.000000	0.000000	0.000000
4332.7	0.000000	0.000000	0.000000	0.000000
4486.8	0.000000	0.000000	0.000000	0.000000
4640.9	0.000007	0.000007	0.000007	0.000007
4795.0	0.000000	0.000000	0.000000	0.000000
4949.1	0.000000	0.000000	0.000000	0.000000
5103.2	0.000001	0.000001	0.000001	0.000001
5257.3	0.000000	0.000000	0.000000	0.000000

Table 14: Raw numerical results of the transfer-angle sensitivity analysis.

Angle(°)	Diff_TPP_vs_God	Diff_TPP_vs_Bat	Diff_TPP_vs_Sun	Mean_Diff
10.0	0.000000	0.000000	0.000000	0.000000
16.7	0.000000	0.000000	0.000000	0.000000
23.3	0.000000	0.000000	0.000000	0.000000
30.0	0.000000	0.000000	0.000000	0.000000
36.7	0.000000	0.000000	0.000000	0.000000
43.3	0.000000	0.000000	0.000000	0.000000
50.0	0.000000	0.000000	0.000000	0.000000
56.7	0.000000	0.000000	0.000000	0.000000
63.3	0.000000	0.000000	0.000000	0.000000
70.0	0.000000	0.000000	0.000000	0.000000
76.7	0.000000	0.000000	0.000000	0.000000
83.3	0.000000	0.000000	0.000000	0.000000
90.0	0.000000	0.000000	0.000000	0.000000
96.7	0.000000	0.000000	0.000000	0.000000
103.3	0.000000	0.000000	0.000000	0.000000
110.0	0.000000	0.000000	0.000000	0.000000
116.7	0.000000	0.000000	0.000000	0.000000
123.3	0.000000	0.000000	0.000000	0.000000
130.0	0.000000	0.000000	0.000000	0.000000
136.7	0.000007	0.000007	0.000007	0.000007
143.3	0.000005	0.000005	0.000005	0.000005
150.0	0.000004	0.000004	0.000004	0.000004
156.7	0.000002	0.000002	0.000002	0.000002
163.3	0.000002	0.000002	0.000002	0.000002
170.0	0.000001	0.000001	0.000001	0.000001

Table 15: Raw numerical results of the near-180° robustness analysis.

Angle(°)	TOF(s)	Valid	Diff_TPP_vs_God	Diff_TPP_vs_Bat	Diff_TPP_vs_Sun	Mean_Diff
160.0	8100.0	YES	0.000002	0.000002	0.000002	0.000002
161.4	8100.0	YES	0.000002	0.000002	0.000002	0.000002
162.7	8100.0	YES	0.000002	0.000002	0.000002	0.000002
164.1	8100.0	YES	0.000002	0.000002	0.000002	0.000002
165.4	8100.0	YES	0.000001	0.000001	0.000001	0.000001
166.8	8100.0	YES	0.000001	0.000001	0.000001	0.000001
168.1	8100.0	YES	0.000001	0.000001	0.000001	0.000001
169.5	8100.0	YES	0.000001	0.000001	0.000001	0.000001
170.9	8100.0	YES	0.000001	0.000001	0.000001	0.000001
172.2	8100.0	YES	0.000001	0.000001	0.000001	0.000001
173.6	8100.0	YES	0.000001	0.000001	0.000001	0.000001
174.9	8100.0	YES	0.000001	0.000001	0.000001	0.000001
176.3	8200.0	YES	0.000001	0.000001	0.000001	0.000001
177.6	8200.0	YES	0.000000	0.000000	0.000000	0.000000
179.0	8200.0	YES	0.000000	0.000000	0.000000	0.000000

B Project Schedule

The schedule summarizes the project workflow from proposal definition to final defense:

Table 16: Team Pilot Project completion schedule.

Phase	**Tasks**	**Timeline 2026**
Start	Project Proposal	04/09
Phase 1	Literature Review	04/10 - 04/24
Phase 2	2-Body Lambert Solver Development and Optimization Algorithm	04/25 - 05/07
Review	**Mid-project progress review**	**05/08**
Phase 3	J_2 Perturbation Modeling and Shooting Method Correction	05/09 - 05/17
Phase 4	Code Analysis and Validation	05/18 - 05/27
End	Project Defense	06/08

C Work Distribution

The final work distribution reflects the responsibilities carried throughout the project:

Table 17: Team Pilot Project work distribution.

Task Category	**Member**	**Final Sub-tasks & Deliverables**
Literature Review	Marwane & Fadlan	Reviewed the literature and identified established Lambert algorithms used for validation.
Initial Development	Erick & Carlos	MATLAB implementation of Lambert transfer problem (solver1).
Mission Optimization	Neama & Erick	MATLAB implementation of fuel transfer optimization (solver2).
Perturbation Modeling	Fadlan & Marwane	MATLAB implementation of J_2 perturbation (solver3).
Validation & Analysis	Carlos & Neama	Numerical error validation, precision control analysis, and scenario visualization.

D Code Availability

The final MATLAB implementation is supplied with this report as an accompanying two packages of code.

Table 18: Files included in the MATLAB code package for three lambert solvers.

File	Purpose
`README.md`	Provides execution instructions, input conventions, and file descriptions.
`main_lambert.m`	Interactive program entry point, input validation, and solver selection.
`lambert_solver.m`	Core two-body Lambert solver using the p-iteration method.
`solver1.m`	Computes the prescribed-time transfer, orbital elements, and velocity increments.
`solver2.m`	Determines the minimum-ΔV transfer time using three optimization methods.
`solver3.m`	Applies the J_2 perturbation correction using RK4, STM propagation, and shooting.
`coe_from_sv.m`	Converts position and velocity vectors of the transfer orbit into classical orbital elements.
`validation_checks.m`	Numerical validation checks (position closure, energy, angular momentum, p-iteration convergence).

Table 19: Files included in the MATLAB code package for the comparative analysis.

File	Purpose
`README.md`	Execution instructions, input conventions, and file descriptions.
`lambert_gooding.m`	Gooding's Lambert solver using Halley's method with sigma function series.
`lambert_battin.m`	Battin's Lambert solver using successive substitution with continued fractions.
`lambert_sun.m`	Sun's Lambert solver using Laguerre iteration with analytic derivatives.
`lambert_tpp.m`	TPP solver1 (p-iteration), adapted to have the same output format as the other solvers for direct comparison.
`lambert_comprehensive_analysis.m`	Main script that runs all test cases (time-of-flight, angle sweep, near-180° cases) and collects metrics.
`lambert_plotting_analysis.m`	Generates all figures from saved results.
`RUN_COMPLETE_ANALYSIS.m`	Driver that calls the comprehensive analysis and plotting scripts in sequence.